

Ensemble Learning

Jefferson Morais

UFPA

- Ao fazer uma pergunta complexa para milhares de pessoas aleatórias e combinar suas respostas, a resposta agregada muitas vezes supera a de um especialista
- Fenômeno conhecido como **"sabedoria das multidões"**
- Da mesma forma, ao combinar as previsões de um grupo de modelos (*ensemble*), geralmente obtemos **previsões melhores** do que com apenas o melhor modelo individual
- O conjunto de modelos é chamado de *ensemble*, e essa técnica é conhecida como **Ensemble Learning**
- Eficiente quando modelos individuais têm desempenho semelhante ou complementar em diferentes aspectos do problema
- Ensemble Learning $\Rightarrow$ **comitê de modelos** (classificadores ou regressores)

• Variedade de Modelos

- Ensemble Learning permite combinar diferentes tipos de modelos, aproveitando suas diferentes capacidades e pontos fortes
- Modelos podem ser de diferentes famílias, como árvores de decisão, modelos lineares, modelos probabilísticos, redes neurais, entre outros

• Redução de *Overfitting*

- Ensemble Learning ajuda a reduzir o *overfitting* ao combinar modelos que se especializam em diferentes aspectos do problema
- Modelos sobreajustados podem ser compensados por modelos menos propensos a *overfitting*, resultando em uma previsão mais robusta

• **Melhoria na Estabilidade**

- Ao usar várias instâncias de treinamento e diferentes algoritmos de aprendizado, o Ensemble Learning torna o modelo mais estável e menos sensível a variações nos dados de entrada
- Isso é especialmente útil em conjuntos de dados pequenos ou ruidosos, onde a estabilidade do modelo é crucial

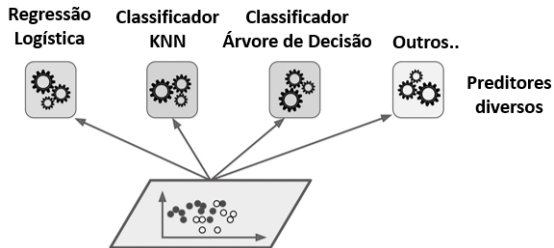
• **Maior Precisão**

- A combinação de previsões de vários modelos geralmente leva a uma predição final mais precisa e confiável do que qualquer modelo individual
- Erros cometidos por um modelo podem ser compensados pelos acertos de outros modelos

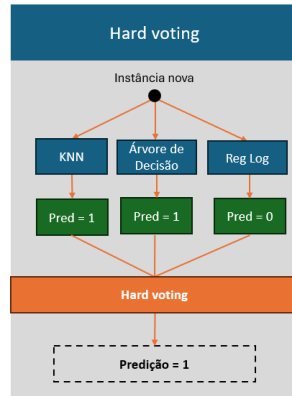
Tipos de Ensemble Learning

- Voting (Averaging)
- Bagging (Bootstrap aggregation) e Pasting
- Random forest
- Boosting
- Stacking

- Referenciado também como classificadores de votação
- Imagine que você tenha treinado diversos classificadores, cada um com uma precisão em torno de 80%
- Entre eles, podem estar incluídos um classificador de Regressão Logística, um K-NN, uma árvore de decisão, um Naive Bayes, e possivelmente alguns outros



- Estratégia para aprimorar um classificador: combinar as previsões de cada classificador e determinar a classe que **recebe a maioria dos votos**
- Esse classificador de previsões de votos majoritários se chama classificador **hard voting**

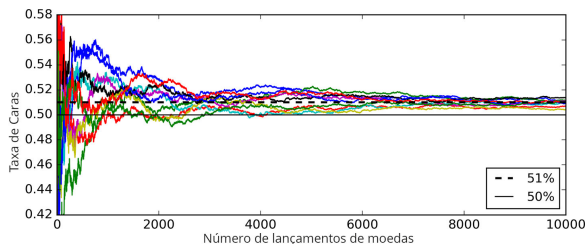


- Curiosamente, este classificador de votação frequentemente atinge uma precisão maior do que o melhor classificador individual no comitê
- Mesmo que cada classificador seja considerado um **aprendiz fraco** (*weak learner*)
 - Ligeiramente melhor do que adivinhação aleatória
- Comitê ainda pode ser um **aprendiz forte** (*strong learner*)
- Desde que haja um **número suficiente** de aprendizes fracos e eles sejam suficientemente diversos

- Mas como isso é possível?

- **Analogia:** Moeda enviesada

- Moeda tem 51% de chance de sair cara e 49% de chance de sair coroa
- Lançada 1.000 vezes, geralmente resulta em 510 caras e 490 coroas
- Probabilidade de obter maioria de caras após 1.000 lançamentos é próxima de 75%
- Quanto mais lançamentos, maior a probabilidade (ex: com 10.000 lançamentos, probabilidade ultrapassa 97%)
- **Lei dos grandes números:** à medida que você continua a jogar a moeda, a proporção de caras se aproxima cada vez mais da probabilidade de caras (51%)



- Agora do mesmo modo considere um comitê de 1.000 classificadores
- Cada um com uma taxa de acerto de apenas 51%
- Usando a classe mais votada, podemos alcançar até 75% de precisão nas predições
- Contudo, isso só vale se todos os classificadores forem perfeitamente independentes
- Na prática, os classificadores geralmente cometem erros semelhantes devido ao treinamento nos mesmos dados
- Isso resulta em muitos votos errados, reduzindo a precisão geral do modelo de votação

- Os métodos de ensemble se destacam quando os **preditores são independentes** entre si
- Uma abordagem para obter essa independência é treinar os classificadores utilizando **algoritmos diferentes**
- Dessa forma, há uma maior probabilidade de que eles cometam **erros de tipos distintos**.
- Essa diversidade contribui para aprimorar a precisão do comitê

Ensemble Learning - Bagging e Pasting

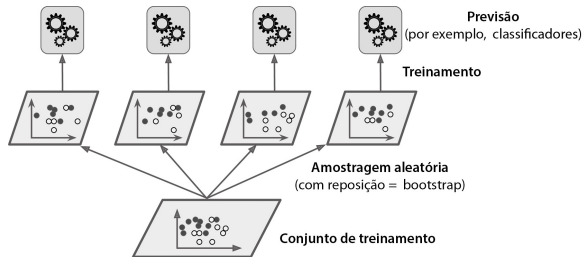
Jefferson Morais

UFPA

- Uma maneira de obter um conjunto diversificado de classificadores é usar algoritmos de treinamento muito diferentes (voting)
- Outra abordagem: usar o **mesmo algoritmo** de treinamento para cada preditor
- mas treiná-los em diferentes **subconjuntos aleatórios** do conjunto de treinamento
- Quando a **amostragem** é realizada **com reposição**, esse método é chamado de **bagging** (abreviação de bootstrap aggregating)
- Quando a **amostragem** é realizada **sem reposição**, é chamada de **pasting**

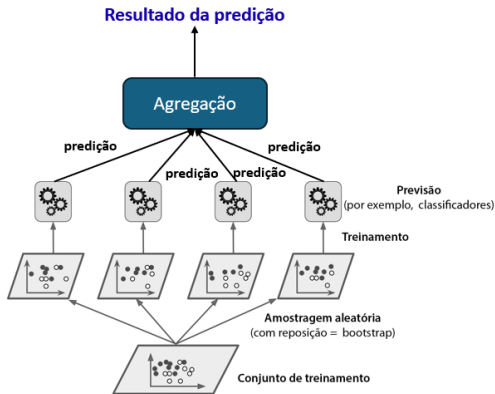
Bagging e Pasting

- Tanto o bagging quanto o pasting permitem que instâncias de treinamento sejam amostradas várias vezes em diferentes preditores
- No entanto, apenas o bagging permite que as mesmas instâncias de treinamento sejam amostradas várias vezes para o mesmo preditor
- Significa que um único preditor pode ver a mesma instância de treinamento mais de uma vez



Bagging - Predição

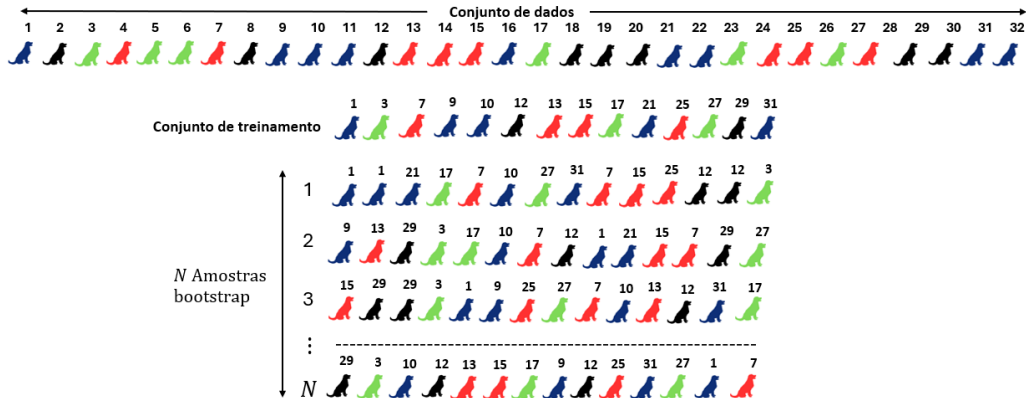
- No processo de predição, o bagging opera da seguinte maneira



- Mas como construir esse método ensemble?

- Com base em um conjunto de treinamento $\mathbf{X}$ contendo m exemplos, o bagging procede de forma iterativa para criar o conjunto
 - 1 Um conjunto de treinamento de tamanho m é criado a partir de $\mathbf{X}$ por meio de amostragem com reposição. Esse processo chamamos de **amostragem bootstrap**
 - 2 Em seguida, um modelo base é construído utilizando os exemplos dessa amostra de bootstrap recém-criada
- Se N é o número de iterações, então N modelos base independentes serão gerados
- Mas o que é Amostragem Bootstrap?

Entendendo Amostragem Bootstrap



- Cada amostra bootstrap consiste em aproximadamente 63.2% de observações únicas do conjunto de dados original

Entendendo Amostragem Bootstrap

- Probabilidade de uma instância ser selecionada em cada replicação da técnica bootstrap

$$1 - \left(1 - \frac{1}{m}\right)^m$$

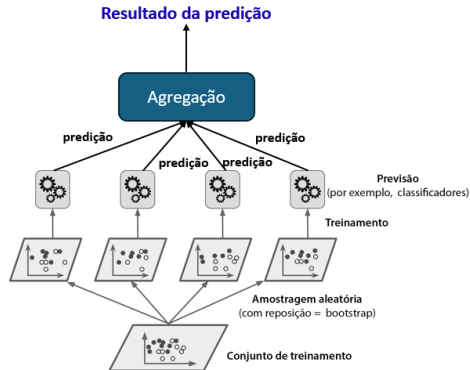
- Em contrapartida, podemos verificar que

$$\lim_{m \rightarrow \infty} \left(1 - \frac{1}{m}\right)^m = \frac{1}{e} \approx 0.368$$

- Para m suficientemente grande, $1 - \frac{1}{e} \approx 0.632$
- Isso significa que, em média, apenas cerca de 63.2% das instâncias de treinamento são amostradas para cada preditor

Bagging - Agregação

- O componente de agregação se comporta de maneira diferente, dependendo do problema de aprendizado
 - No caso de um problema de classificação, ele prevê a moda (ou seja, a classe produzida com mais frequência pelos preditores)
 - Para problemas de regressão, ele prevê a média (ou seja, o valor médio) das previsões produzidas pelos preditores



Algoritmo AdaBoost

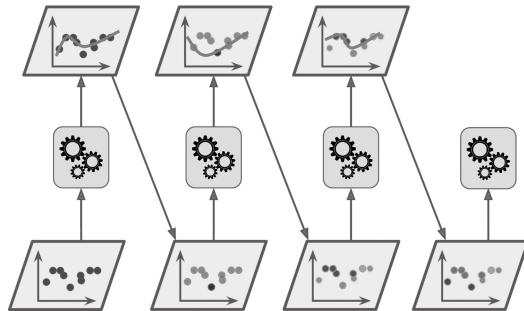
Jefferson Morais

UFPA

- Abreviação de Adaptive Boosting foi proposto por Yoav Freund e Robert Schapire em 1996 ¹
- Funciona treinando uma sequência de classificadores fracos em que cada classificador se concentra nos exemplos que foram classificados incorretamente pelos classificadores anteriores
- Durante o treinamento, cada instância de treinamento é ponderada de acordo com a precisão dos classificadores anteriores

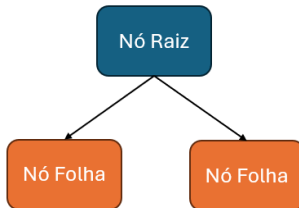
¹<https://doi.org/10.1006/jcss.1997.1504>

- Ex: ao treinar um classificador AdaBoost
 - Algoritmo primeiro treina um *weak learner* (aprendiz fraco) e o utiliza para fazer previsões no conjunto de treinamento
 - O algoritmo então aumenta o peso relativo das instâncias de treinamento classificadas erroneamente
 - Em seguida, ele treina um segundo classificador, usando os pesos atualizados, e novamente faz previsões no conjunto de treinamento, atualiza os pesos das instâncias, e assim por diante



Decision Stump

- No Adaboost, um aprendiz fraco bastante utilizado é o **Decision Stump**
- Decision Stump é essencialmente uma árvore de decisão com apenas um nível
 - Árvore de decisão que faz uma única divisão com base em uma única característica



Outros aprendizes fracos podem ser utilizados tais como regressores lineares, modelos lineares generalizados, KNN, etc.

AdaBoost - Algoritmo

- Dado o conjunto de Treinamento $\mathbf{X} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\}$, onde $(\mathbf{x}, y)$ é uma instância de $\mathbf{X}$ e y é o rótulo correspondente (+1 ou -1).
- Inicialize os pesos das instâncias de treinamento, $w^{(i)} = \frac{1}{m}$, para $i = 1, 2, \dots, m$.
- **Para cada iteração** $t = 1, 2, \dots, T$, onde T é o número de aprendiz fracos
 - Treine um modelo fraco $h_t(x)$ a partir do *TrainSet* $\mathbf{X}_t$ obtido pela amostragem de $\mathbf{X}$ de acordo com o peso
 - Calcule o erro ponderado ϵ_t do modelo fraco h_t :

$$\epsilon_t = \frac{\sum_{i=1}^m w^{(i)} \cdot I(h_t(x^{(i)}) \neq y^{(i)})}{\sum_{i=1}^m w^{(i)}}$$

- Calcule o peso α_t do modelo fraco:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

- Atualize os pesos das instâncias de treinamento:

$$w^{(i)} \leftarrow w^{(i)} \cdot \exp(-\alpha_t \cdot y^{(i)} \cdot h_t(x^{(i)})) \text{ para } i = 1, 2, \dots, m$$

- Normalize os pesos $w^{(i)}$ para que somem 1 (dividir por $\sum_{i=1}^m w^{(i)}$)
- Combine os modelos fracos ponderados para formar o modelo final:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$$

- $I(\cdot)$ é a função indicadora que retorna 1 se a condição for verdadeira e 0 caso contrário
- $\text{sign}(\cdot)$ é a função de sinal que retorna +1 se o argumento for maior ou igual a zero e -1 caso contrário

AdaBoost - Exemplo

- Considere a aplicação do algoritmo AdaBoost em um conjunto de dados para prever se um jogo será jogado ou não com base nas condições meteorológicas
- Abaixo o conjunto de dados

Temperatura	Vento	Umidade	Jogar
Alta	Não	Sim	-1
Baixa	Sim	Não	1
Alta	Sim	Sim	-1
Alta	Não	Sim	1
Baixa	Sim	Sim	1
Alta	Sim	Não	-1

- Objetivo: Construir um modelo de predição usando o AdaBoost com 3 classificadores **decision stump**
- -1 indica jogar **Não** e 1 indica jogar **Sim**
- Determinar a predição final para: Temperatura = Alta, Vento = Não, Umidade = Não

- Algoritmo AdaBoost - Exemplo

Algoritmo AdaBoost - Exemplo

Jefferson Moraes

UFPA

AdaBoost - Algoritmo

- Dado o conjunto de Treinamento $\mathbf{X} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\}$, onde $(\mathbf{x}, y)$ é uma instância de $\mathbf{X}$ e y é o rótulo correspondente (+1 ou -1).
- Inicialize os pesos das instâncias de treinamento, $w^{(i)} = \frac{1}{m}$, para $i = 1, 2, \dots, m$.
- **Para cada iteração** $t = 1, 2, \dots, T$, onde T é o número de aprendiz fracos
 - Treine um modelo fraco $h_t(x)$ a partir do *TrainSet* $\mathbf{X}_t$ obtido pela amostragem de $\mathbf{X}$ de acordo com o peso
 - Calcule o erro ponderado ϵ_t do modelo fraco h_t :

$$\epsilon_t = \frac{\sum_{i=1}^m w^{(i)} \cdot I(h_t(x^{(i)}) \neq y^{(i)})}{\sum_{i=1}^m w^{(i)}}$$

- Calcule o peso α_t do modelo fraco:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

- Atualize os pesos das instâncias de treinamento:

$$w^{(i)} \leftarrow w^{(i)} \cdot \exp(-\alpha_t \cdot y^{(i)} \cdot h_t(x^{(i)})) \text{ para } i = 1, 2, \dots, m$$

- Normalize os pesos $w^{(i)}$ para que somem 1 (dividir por $\sum_{i=1}^m w^{(i)}$)
- Combine os modelos fracos ponderados para formar o modelo final:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$$

- $I(\cdot)$ é a função indicadora que retorna 1 se a condição for verdadeira e 0 caso contrário
- $\text{sign}(\cdot)$ é a função de sinal que retorna +1 se o argumento for maior ou igual a zero e -1 caso contrário

AdaBoost - Algoritmo (corrigido)

- Dado o conjunto de Treinamento $\mathbf{X} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\}$, onde $(\mathbf{x}, y)$ é uma instância de $\mathbf{X}$ e y é o rótulo correspondente (+1 ou -1).
- Inicialize os pesos das instâncias de treinamento, $w^{(i)} = \frac{1}{m}$, para $i = 1, 2, \dots, m$.
- **Para cada iteração** $t = 1, 2, \dots, T$, onde T é o número de aprendizes fracos
 - Treine um modelo fraco $h_t(x)$ a partir do *TrainSet* $\mathbf{X}_t$ obtido pela amostragem de $\mathbf{X}$ de acordo com o peso
 - Calcule o erro ponderado ϵ_t do modelo fraco h_t : $\epsilon_t = \sum_{i=1}^m w^{(i)} \cdot I(h_t(\mathbf{x}^{(i)}) \neq y^{(i)})$
 - Calcule o peso α_t do modelo fraco:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

- Atualize os pesos das instâncias de treinamento:

$$w^{(i)} \leftarrow w^{(i)} \cdot \exp(-\alpha_t \cdot y^{(i)} \cdot h_t(\mathbf{x}^{(i)})) \text{ para } i = 1, 2, \dots, m$$

- Normalize os pesos $w^{(i)}$ para que somem 1 (dividir por $\sum_{i=1}^m w^{(i)}$)
- Combine os modelos fracos ponderados para formar o modelo final:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$$

- $I(\cdot)$ é a função indicadora que retorna 1 se a condição for verdadeira e 0 caso contrário
- $\text{sign}(\cdot)$ é a função de sinal que retorna +1 se o argumento for maior ou igual a zero e -1 caso contrário

AdaBoost - Exemplo

- Considere a aplicação do algoritmo AdaBoost em um conjunto de dados para prever se um jogo será jogado ou não com base nas condições meteorológicas
- Abaixo o conjunto de dados

Temperatura	Vento	Umidade	Jogar
Alta	Não	Sim	-1
Baixa	Sim	Não	1
Alta	Sim	Sim	-1
Alta	Não	Sim	1
Baixa	Sim	Sim	1
Alta	Sim	Não	-1

- Objetivo: Construir um modelo de predição usando o AdaBoost com 3 classificadores **decision stump**
- -1 indica jogar **Não** e 1 indica jogar **Sim**
- Determinar a predição final para: Temperatura = Alta, Vento = Não, Umidade = Não

AdaBoost - Exemplo

- Passo 1: Inicialize os pesos das instâncias de treinamento, $w^{(i)} = \frac{1}{m}$, para $i = 1, 2, \dots, m$

Temperatura	Vento	Umidade	Jogar	pesos (w)
Alta	Não	Sim	-1	1/6
Baixa	Sim	Não	1	1/6
Alta	Sim	Sim	-1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6
Alta	Sim	Não	-1	1/6

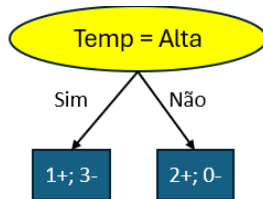
- **Passo 2:** Treinamento dos classificadores fracos

- Considere $T = 3$ classificadores fracos, ou seja, vamos repetir o processo de $t = 1, 2, 3$
- **Passo 2.1** Treine um modelo fraco $h_1(x)$ a partir do *TrainSet* $\mathbf{X}_1$ obtido pela amostragem de $\mathbf{X}$ de acordo com o peso
- Para o exemplo vamos usar como aprendiz fraco um Decision Stump (Árvore de Decisão com profundidade 1)
- Então, vamos escolher nosso primeiro Decision Stump

AdaBoost - Exemplo

- Escolhendo o primeiro aprendiz fraco $h_1(x)$

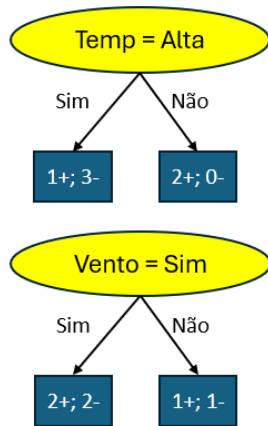
Temperatura	Vento	Umidade	Jogar	pesos (w)
Alta	Não	Sim	-1	1/6
Baixa	Sim	Não	1	1/6
Alta	Sim	Sim	-1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6
Alta	Sim	Não	-1	1/6



AdaBoost - Exemplo

- Escolhendo o primeiro aprendiz fraco $h_1(x)$

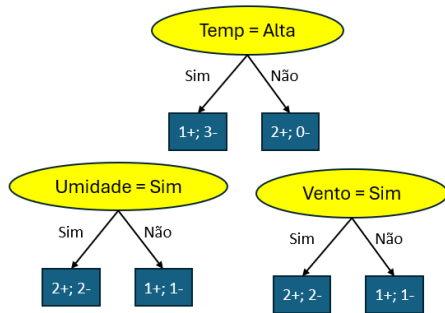
Temperatura	Vento	Umidade	Jogar	pesos (w)
Alta	Não	Sim	-1	1/6
Baixa	Sim	Não	1	1/6
Alta	Sim	Sim	-1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6
Alta	Sim	Não	-1	1/6



AdaBoost - Exemplo

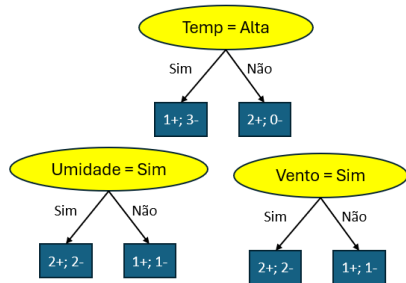
- Escolhendo o primeiro aprendiz fraco $h_1(x)$

Temperatura	Vento	Umidade	Jogar	pesos (w)
Alta	Não	Sim	-1	1/6
Baixa	Sim	Não	1	1/6
Alta	Sim	Sim	-1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6
Alta	Sim	Não	-1	1/6



AdaBoost - Exemplo

- Escolhendo o primeiro aprendiz fraco $h_1(x)$



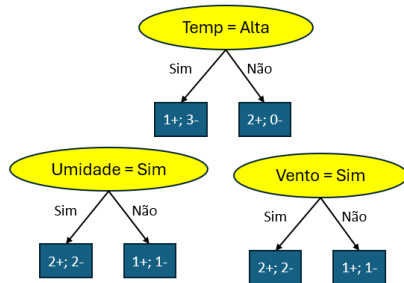
- Para avaliar cada um vamos usar o Ganho de Informação com Índice Gini (classificação binária)

$$\text{Gini}(t) = 2p(1 - p)$$

- Logo, antes do split temos: $\text{Gini}(\text{Temp}) = 0.5$
- Para $\text{Temp} = \text{Alta}$
 - $\text{Gini}(\text{temp}=\text{sim}) = 2 * 1/4(1 - 1/4) = 0.375$
 - $\text{Gini}(\text{temp}=\text{não}) = 0$
 - $IG(\text{temp}) = 0.5 - ((4/6) * 0.375 + (2/6) * 0) = 0.25$

AdaBoost - Exemplo

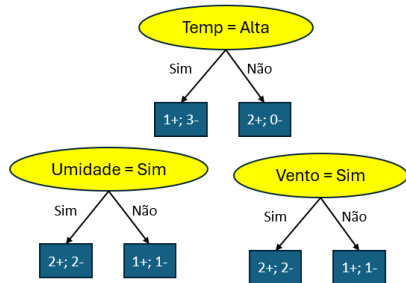
- Escolhendo o primeiro aprendiz fraco $h_1(x)$



- Para Vento = sim
 - $Gini(vento=sim) = 0.5$
 - $Gini(vento=não) = 0.5$
 - $IG(vento) = 0$

AdaBoost - Exemplo

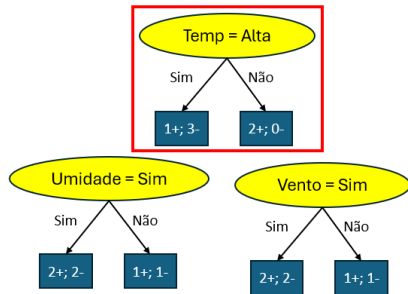
- Escolhendo o primeiro aprendiz fraco $h_1(x)$



- Para Umidade = Sim
 - $Gini(Umidade=sim) = 0.5$
 - $Gini(vento=não) = 0.5$
 - $IG(vento) = 0$

AdaBoost - Exemplo

- Escolhendo o primeiro aprendiz fraco $h_1(x)$



- Considerando
 - $IG(\text{temp}=\text{alta}) = 0.25$
 - $IG(\text{vento}=\text{sim}) = 0$
 - $IG(\text{umidade}=\text{sim}) = 0$
- Temos que o aprendiz escolhido é o **Temp = Alta**, logo $h_1 = \text{Temp}=\text{Alta}$

AdaBoost - Exemplo

- **Passo 1.2:** Calcule o erro ponderado ϵ_1 do modelo fraco h_1 :

$$\epsilon_1 = \sum_{i=1}^m w^{(i)} \cdot I(h_1(x^{(i)}) \neq y^{(i)})$$

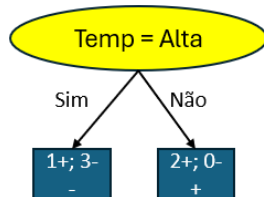
$$\epsilon_1 = 1/6$$

- Calcule o peso α_1 do modelo fraco:

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - \epsilon_1}{\epsilon_1} \right)$$

$$\alpha_1 = 1/2 \ln \left(\frac{1 - 1/6}{1/6} \right) = 1/2 * \ln(5) \approx 0.80$$

Temperatura	Vento	Umidade	Jogar	pesos (w)
Alta	Não	Sim	-1	1/6
Baixa	Sim	Não	1	1/6
Alta	Sim	Sim	-1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6
Alta	Sim	Não	-1	1/6



AdaBoost - Exemplo

- **Passo 1.3:** Atualize os pesos das instâncias de treinamento:

$$w^{(i)} \leftarrow w^{(i)} \cdot \exp(-\alpha_t \cdot y^{(i)} \cdot h_t(x^{(i)}))$$

- $\alpha_1 = 0.80$

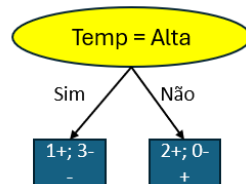
- “Inst. classificadas **corretamente**”

$$1/6 * \exp(-0.80 * 1) = 0.07$$

- “Inst. Classificadas **incorretamente**”

$$1/6 * \exp(-0.8 * -1) = 0.37$$

Temp.	Vento	Umidade	Jogar	pesos	pesos ajustados
Alta	Não	Sim	-1	1/6	0.07
Baixa	Sim	Não	1	1/6	0.07
Alta	Sim	Sim	-1	1/6	0.07
Alta	Não	Sim	1	1/6	0.37
Baixa	Sim	Sim	1	1/6	0.07
Alta	Sim	Não	-1	1/6	0.07



AdaBoost - Exemplo

- **Passo 1.4:** Normalização dos Pesos
- Dividir os pesos novos por $\sum_{i=1}^m w^{(i)}$

$$\Sigma_{\text{pesos}} = 5 * 0.07 + 0.032 = 0.72$$

Temp.	Vento	Umidade	Jogar	pesos	novos pesos	novos pesos normalizados
Alta	Não	Sim	-1	1/6	0.07	0.098
Baixa	Sim	Não	1	1/6	0.07	0.098
Alta	Sim	Sim	-1	1/6	0.07	0.098
Alta	Não	Sim	1	1/6	0.37	0.51
Baixa	Sim	Sim	1	1/6	0.07	0.098
Alta	Sim	Não	-1	1/6	0.07	0.098

AdaBoost - Exemplo

- Repetindo o Algoritmo agora para **segunda iteração**

- **Passo 2.1:** Treinando do modelo $h_2(x)$ a partir do novo trainSet $\mathbf{X}_2$ obtido pela amostragem considerando os pesos das instâncias

- Base anterior

Temp.	Vento	Umidade	Jogar	pesos	intervalo
Alta	Não	Sim	-1	0.098	[0, 0.098)
Baixa	Sim	Não	1	0.098	[0.098, 0.196)
Alta	Sim	Sim	-1	0.098	[0.196, 0.294)
Alta	Não	Sim	1	0.51	[0.294, 0.804)
Baixa	Sim	Sim	1	0.098	[0.804, 0.902)
Alta	Sim	Não	-1	0.098	[0.902, 1.0]

- Geração de número aleatório $[0, 1] \rightarrow 0.931$
- Escolhe a Instância: 6
- Base $\mathbf{X}_2$

Temp.	Vento	Umidade	Jogar
Alta	Sim	Não	-1

AdaBoost - Exemplo

- Repetindo o Algoritmo agora para **segunda iteração**

- **Passo 2.1:** Treinando do modelo $h_2(x)$ a partir do novo trainSet $\mathbf{X}_2$ obtido pela amostragem considerando os pesos das instâncias

- Base anterior

- Geração de número aleatório $[0, 1] \rightarrow 0.782$
- Escolhe a Instância: 4
- Base $\mathbf{X}_2$

Temp.	Vento	Umidade	Jogar	pesos	intervalo
Alta	Não	Sim	-1	0.098	$[0, 0.098)$
Baixa	Sim	Não	1	0.098	$[0.098, 0.196)$
Alta	Sim	Sim	-1	0.098	$[0.196, 0.294)$
Alta	Não	Sim	1	0.51	$[0.294, 0.804)$
Baixa	Sim	Sim	1	0.098	$[0.804, 0.902)$
Alta	Sim	Não	-1	0.098	$[0.902, 1.0]$

Temp.	Vento	Umidade	Jogar
Alta	Sim	Não	-1
Alta	Não	Sim	1

AdaBoost - Exemplo

- Repetindo o Algoritmo agora para **segunda iteração**

- **Passo 2.1:** Treinando do modelo $h_2(x)$ a partir do novo trainSet $\mathbf{X}_2$ obtido pela amostragem considerando os pesos das instâncias

- Base anterior

Temp.	Vento	Umidade	Jogar	pesos	intervalo
Alta	Não	Sim	-1	0.098	$[0, 0.098)$
Baixa	Sim	Não	1	0.098	$[0.098, 0.196)$
Alta	Sim	Sim	-1	0.098	$[0.196, 0.294)$
Alta	Não	Sim	1	0.51	$[0.294, 0.804)$
Baixa	Sim	Sim	1	0.098	$[0.804, 0.902)$
Alta	Sim	Não	-1	0.098	$[0.902, 1.0]$

- Geração de número aleatório $[0, 1] \rightarrow 0.325$
- Escolhe a Instância: 4
- Base $\mathbf{X}_2$

Temp.	Vento	Umidade	Jogar
Alta	Sim	Não	-1
Alta	Não	Sim	1
Alta	Não	Sim	1

AdaBoost - Exemplo

- Repetindo o Algoritmo agora para **segunda iteração**

- **Passo 2.1:** Treinando do modelo $h_2(x)$ a partir do novo trainSet $\mathbf{X}_2$ obtido pela amostragem considerando os pesos das instâncias

- Base anterior

Temp.	Vento	Umidade	Jogar	pesos	intervalo
Alta	Não	Sim	-1	0.098	$[0, 0.098)$
Baixa	Sim	Não	1	0.098	$[0.098, 0.196)$
Alta	Sim	Sim	-1	0.098	$[0.196, 0.294)$
Alta	Não	Sim	1	0.51	$[0.294, 0.804)$
Baixa	Sim	Sim	1	0.098	$[0.804, 0.902)$
Alta	Sim	Não	-1	0.098	$[0.902, 1.0]$

- Geração de número aleatório $[0, 1] \rightarrow 0.176$
- Escolhe a Instância: 2
- Base $\mathbf{X}_2$

Temp.	Vento	Umidade	Jogar
Alta	Sim	Não	-1
Alta	Não	Sim	1
Alta	Não	Sim	1
Baixa	Sim	Não	1

AdaBoost - Exemplo

- Repetindo o Algoritmo agora para **segunda iteração**
- **Passo 2.1:** Treinando do modelo $h_2(x)$ a partir do novo trainSet $\mathbf{X}_2$ obtido pela amostragem considerando os pesos das instâncias
- Base anterior

Temp.	Vento	Umidade	Jogar	pesos	intervalo
Alta	Não	Sim	-1	0.098	$[0, 0.098)$
Baixa	Sim	Não	1	0.098	$[0.098, 0.196)$
Alta	Sim	Sim	-1	0.098	$[0.196, 0.294)$
Alta	Não	Sim	1	0.51	$[0.294, 0.804)$
Baixa	Sim	Sim	1	0.098	$[0.804, 0.902)$
Alta	Sim	Não	-1	0.098	$[0.902, 1.0]$

- Geração de número aleatório $[0, 1] \rightarrow 0.546$
- Escolhe a Instância: 4
- Base $\mathbf{X}_2$

Temp.	Vento	Umidade	Jogar
Alta	Sim	Não	-1
Alta	Não	Sim	1
Alta	Não	Sim	1
Baixa	Sim	Não	1
Alta	Não	Sim	1

AdaBoost - Exemplo

- Repetindo o Algoritmo agora para **segunda iteração**
- **Passo 2.1:** Treinando do modelo $h_2(x)$ a partir do novo trainSet $\mathbf{X}_2$ obtido pela amostragem considerando os pesos das instâncias
- Base anterior

Temp.	Vento	Umidade	Jogar	pesos	intervalo
Alta	Não	Sim	-1	0.098	$[0, 0.098)$
Baixa	Sim	Não	1	0.098	$[0.098, 0.196)$
Alta	Sim	Sim	-1	0.098	$[0.196, 0.294)$
Alta	Não	Sim	1	0.51	$[0.294, 0.804)$
Baixa	Sim	Sim	1	0.098	$[0.804, 0.902)$
Alta	Sim	Não	-1	0.098	$[0.902, 1.0]$

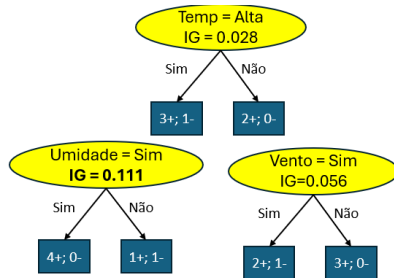
- Geração de número aleatório $[0, 1] \rightarrow 0.847$
- Escolhe a Instância: 5
- Base $\mathbf{X}_2$

Temp.	Vento	Umidade	Jogar	pesos iniciais
Alta	Sim	Não	-1	1/6
Alta	Não	Sim	1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Não	1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6

AdaBoost - Exemplo

- Escolhendo o **segundo** aprendiz fraco $h_2(x)$

Temp.	Vento	Umidade	Jogar	pesos iniciais
Alta	Sim	Não	-1	1/6
Alta	Não	Sim	1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Não	1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6

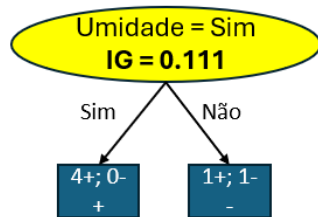


AdaBoost - Exemplo

- **Passo 2.2:** erro ponderado ϵ_2 e peso α_2 do modelo fraco h_2

- $\epsilon_2 = 1/6$
- $\alpha_2 = 0.80$

Temp.	Vento	Umidade	Jogar	pesos iniciais
Alta	Sim	Não	-1	1/6
Alta	Não	Sim	1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Não	1	1/6
Alta	Não	Sim	1	1/6
Baixa	Sim	Sim	1	1/6



- **Passos 2.3 e 2.4:** Atualização e normalização dos pesos
- Como o $\alpha_1 = \alpha_2 = 0.80$, os pesos também serão iguais

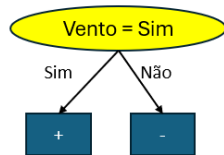
Temp.	Vento	Umidade	Jogar	novos pesos
Alta	Sim	Não	-1	0.098
Alta	Não	Sim	1	0.098
Alta	Não	Sim	1	0.098
Baixa	Sim	Não	1	0.51
Alta	Não	Sim	1	0.098
Baixa	Sim	Sim	1	0.098

AdaBoost - Exemplo

- Repetindo o Algoritmo agora para **terceira e última iteração**
- **Passo 3.1:** Treinando do modelo $h_3(x)$ a partir do novo trainSet $\mathbf{X}_3$ obtido pela amostragem dos dados anteriores considerando os pesos das instâncias
- Base anterior

Temp.	Vento	Umidade	Jogar	pesos	intervalo
Alta	Sim	Não	-1	0.098	$[0, 0.098)$
Alta	Não	Sim	1	0.098	$[0.098, 0.196)$
Alta	Não	Sim	1	0.098	$[0.196, 0.294)$
Baixa	Sim	Não	1	0.51	$[0.294, 0.804)$
Alta	Não	Sim	1	0.098	$[0.804, 0.902)$
Baixa	Sim	Sim	1	0.098	$[0.902, 1.0]$

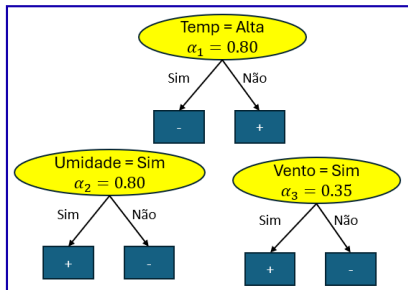
- Suponha que para esta última iteração
 - Foi gerada uma base $\mathbf{X}_3$
 - O aprendiz fraco $h_3(x)$ escolhido foi



- $\epsilon_3 = 2/6 = 1/3$ e $\alpha_3 = 0.35$

AdaBoost - Exemplo

- Considerando o modelo final



- Determinar a predição final para: **Temperatura = Alta, Vento = Não, Umidade = Não**

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$$

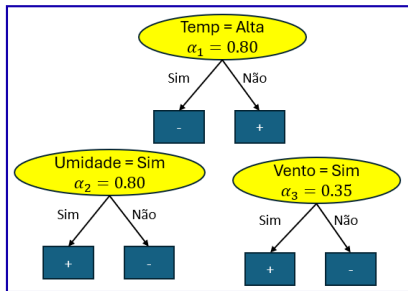
$$H(X) = \text{sign}(0.8 * (-1) + 0.8 * (-1) + 0.35 * (-1))$$

$$H(X) = \text{sign}(-0.8 - 0.8 - 0.35) = \text{sign}(-1.95) = -1$$

- Logo, o Algoritmo AdaBoost classifica a instância como -1, ou seja, jogar não.

AdaBoost - Exemplo

- Considerando o modelo final



- Outro exemplo: **Temperatura = Baixa, Vento = Sim, Umidade = Não**

$$H(X) = \text{sign}(0.8 * 1 + 0.8 * (-1) + 0.35 * 1)$$

$$H(X) = \text{sign}(0.8 - 0.8 + 0.35) = \text{sign}(+0.35) = +1$$

- Logo, o Algoritmo AdaBoost classifica a instância como +1, ou seja, jogar sim.

Gradiente Boosting

Jefferson Morais

UFPA

- **Parte I**

- Introdução e Breve Histórico
- Diferença entre AdaBoost e Gradiente Boosting

- Parte II - Algoritmo do Gradiente Boosting em Regressão

- Parte III - Algoritmo do Gradiente Boosting em Classificação

- Gradiente Boosting é um método ensemble learning (comitê de classificadores)
- Combina vários **modelos fracos** em um único **modelo forte** e preciso
- Cada modelo fraco (ex.: decision stump), é treinado **sequencialmente** para focar nas áreas onde o modelo geral apresenta **maior erro**
- Permite que o Gradiente Boosting aprenda com mais eficiência e obtenha **melhor desempenho** em comparação com modelos individuais
- Tornou-se uma das abordagens **mais populares** em problemas de regressão e classificação

- A história do Gradiente Boosting remonta ao final dos anos 1990
- Jerome H. Friedman e equipe introduziram o algoritmo GBM em seu artigo “Additive Logistic Regression: A Statistical View of Boosting” em 2000 ¹
- Outras implementações e variantes do GBM
 - XGBoost (2014): Desenvolvido por Tianqi Chen, introduziu várias inovações, incluindo algoritmos eficientes para a construção de árvores de decisão e técnicas avançadas de regularização
 - LightGBM (2016): Desenvolvido pela Microsoft, destaca-se por sua eficiência em termos de memória e velocidade de treinamento
 - CatBoost (2017): Desenvolvido pelo Yandex, lida de forma eficaz com variáveis categóricas sem a necessidade de pré-processamento

¹<https://hastie.su.domains/Papers/AdditiveLogisticRegression/alr.pdf>

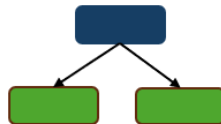
AdaBoost x Gradiente Boosting

- Existem muitas similaridades entre AdaBoost e Gradiente Boosting
- Considere o seguinte conjunto de dados de exemplo, com informações sobre preços de habitação

Área (m ²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000

AdaBoost x Gradiente Boosting

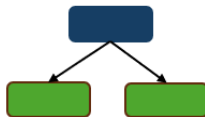
Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



AdaBoost inicia criando um decision stump a partir do conjunto de treinamento

AdaBoost x Gradiente Boosting

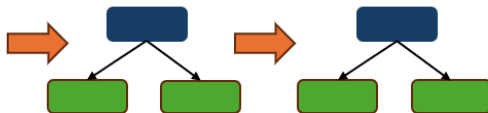
Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



e o peso de influência que o stump tem na saída depende de quão bem ele compensou esses erros anteriores

AdaBoost x Gradiente Boosting

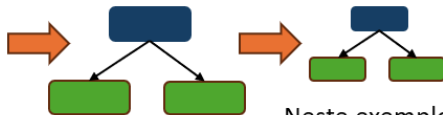
Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



Assim, AdaBoost constrói o próximo stump baseado nos erros gerados pelo stump anterior

AdaBoost x Gradiente Boosting

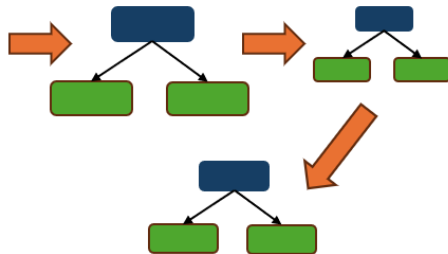
Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



Neste exemplo, o novo stump fez um trabalho ruim compensando os erros do stump anterior e seu tamanho reflete seu reduzido peso de influência

AdaBoost x Gradiente Boosting

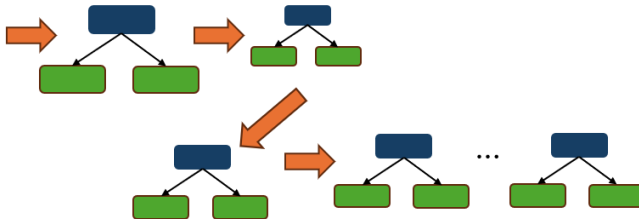
Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



Agora AdaBoost constrói outro stump com base nos erros cometidos pelo stump anterior e este stump teve um desempenho um pouco melhor do que o último stump, por isso é um pouco maior.

AdaBoost x Gradiente Boosting

Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



AdaBoost continua criando stumps dessa maneira até que tenha criado o número de stumps solicitado, ou até que tenha um ajuste perfeito

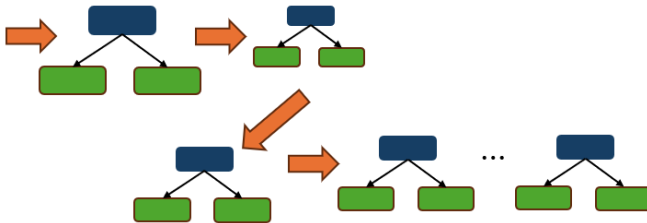
AdaBoost x Gradiente Boosting

Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



Em contraste, o Gradient Boosting começa fazendo uma única folha, em vez de um stump.

Esta folha representa um palpite inicial para os **preços** de todas as amostras



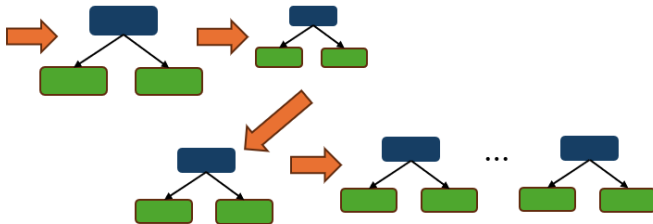
AdaBoost x Gradiente Boosting

Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



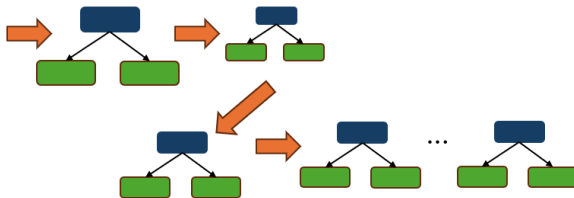
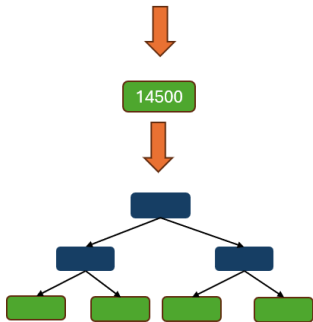
14500

Ao tentar **predizer** um valor contínuo como o **preço**, o primeiro palpite é o valor **médio**



AdaBoost x Gradiente Boosting

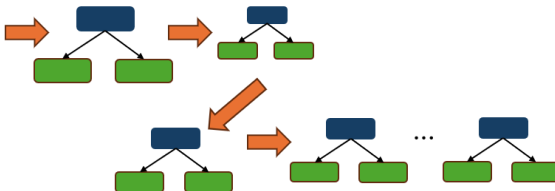
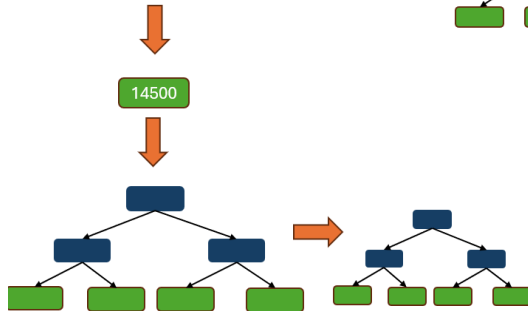
Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



- Depois o **Gradiente Boosting** constrói uma árvore
- Como AdaBoost, essa árvore é baseada nos erros cometidos pelas árvores anteriores
- Entretanto, essa árvore é geralmente maior que um stump
- Gradient Boosting ainda restringe o tamanho da árvore
- Vamos trabalhar com no máximo 4 folhas, mas na prática esse número varia entre 8 e 32

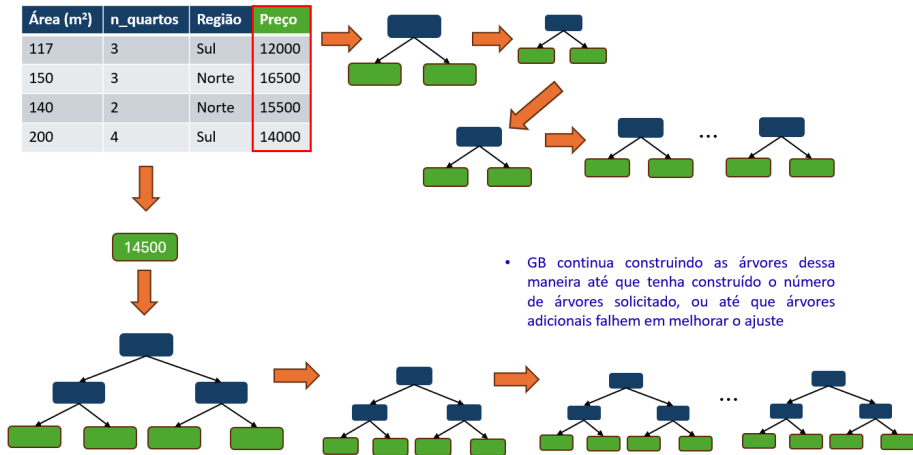
AdaBoost x Gradiente Boosting

Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000



- Assim como o AdaBoost, o Gradiente Boosting constrói árvores de tamanho fixo com base nos erros da árvore anterior, mas ao contrário do AdaBoost, cada árvore pode ser maior que um stump
- Além disso, assim como o AdaBoost, o Gradiente Boosting dimensiona as árvores. No entanto, o Gradiente Boosting dimensiona todas as árvores pela mesma quantidade
- Em seguida, o Gradiente Boosting constrói outra árvore com base nos erros cometidos pela árvore anterior e, em seguida, dimensiona a árvore

AdaBoost x Gradiente Boosting



- Resumo das diferenças chave entre AdaBoost e Gradiente Boosting
 - **Abordagem de Ponderação:** AdaBoost atribui pesos às instâncias de dados, enquanto o Gradiente Boosting ajusta os modelos aos resíduos dos modelos anteriores
 - **Funcionamento Interno:** AdaBoost se concentra em identificar os exemplos difíceis de classificar, enquanto o Gradiente Boosting busca minimizar diretamente uma função de perda diferenciável
 - **Convergência:** AdaBoost pode ser mais suscetível ao overfitting se não forem impostas restrições nos modelos fracos, enquanto o Gradiente Boosting tende a ser mais robusto
 - **Flexibilidade:** O Gradiente Boosting oferece mais flexibilidade na escolha da função de perda e na estrutura dos modelos fracos

- Algoritmo do Gradiente Boosting em Regressão

Algoritmo do Gradiente Boosting em Regressão

Jefferson Moraes

Gestorlearn

Algoritmo do Gradiente Boosting

- **Entrada:** Conjunto de Dados $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, onde $y^{(i)} \in \mathbb{R}$ e uma função de perda (*loss function*) diferenciável $\mathcal{L}(y^{(i)}, F(\mathbf{x}))$

- **Passo 1:** Inicialize o modelo com um valor constante: $F_0(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^m \mathcal{L}(y^{(i)}, \gamma)$

- **Passo 2:** Para $t = 1$ até T :

- **2.1** Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})} \right]_{F(\mathbf{x})=F_{t-1}(\mathbf{x})} \quad \text{para } i = 1, \dots, m$$

- **2.2** Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$

- **2.3** Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{\mathbf{x}^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(\mathbf{x}^{(i)}) + \gamma)$$

- **2.4** Atualize o modelo $F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathcal{I}[\mathbf{x} \in R_{jt}]$

- Retorne $F_T(\mathbf{x})$

- **Entrada:** Conjunto de Dados $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, onde $y^{(i)} \in \mathbb{R}$ e uma função de perda (*loss function*) diferenciável $\mathcal{L}(y^{(i)}, F(\mathbf{x}))$

- Abstraímos a descrição do conjunto de dados de treinamento

Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000

- No exemplo $m = 4$

- Uma função de perda $\mathcal{L}$ é apenas uma medida que avalia o quão precisamente podemos prever o preço

- A função de perda mais frequente usada durante a regressão com Gradiente Boosting é

$$\frac{1}{2}(\text{Observado} - \text{Predito})^2$$

- Assim, a função de perda é simplesmente um resíduo ao quadrado

Passo 1: Inicializar o modelo

- Vamos chamar de F_0 a primeira versão do modelo de ensemble
- Para construir F_0 , o Gradiente Boosting começa induzindo uma função constante a partir dos dados de treinamento, ou seja, $F_0(x) = \gamma$, para todo x
- O valor da constante γ é calculado como a média da variável alvo (**preço**)

- Matematicamente temos

$$F_0(x) = \arg \min_{\gamma} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(y^{(i)}, \gamma)$$

- Mas como é que γ é uma média?

Passo 1: Inicializar o modelo

- GB calcula o valor de γ que minimiza a função de perda $\mathcal{L}$
- Considerando a função de perda como resíduo ao quadrado $1/2(\text{observado} - \text{predito})^2$ ou

$$\mathcal{L}(y^{(i)}, \gamma) = \frac{1}{2}(y^{(i)} - \gamma)^2$$

- Assim, queremos encontrar γ que minimize

$$\sum_{i=1}^m \mathcal{L}(y^{(i)}, \gamma) = \sum_{i=1}^m \frac{1}{2}(y^{(i)} - \gamma)^2 = \frac{1}{2} \sum_{i=1}^m (y^{(i)} - \gamma)^2$$

- Onde

- γ é o valor constante que precisamos calcular
- m é o número de exemplos de treinamento
- $y^{(i)}$ é o valor alvo para o i -ésimo exemplo de treinamento;
- $\mathcal{L}$ é a função de perda que escolhemos

Passo 1: Inicializar o modelo

- Vamos provar que valor de γ é média da variável de saída

$$\frac{d}{d\gamma} \left[\frac{1}{2} \sum_{i=1}^m (y^{(i)} - \gamma)^2 \right] = \frac{1}{2} \sum_{i=1}^m \frac{d}{d\gamma} [(y^{(i)} - \gamma)^2]$$

- Para encontrar a derivada dessa expressão $(y^{(i)} - \gamma)$ em relação a γ , aplicamos a **regra da cadeia**
- A expressão $(y^{(i)} - \gamma)^2$ é uma função composta, onde $y^{(i)} - \gamma$ é a função interna e $(\cdot)^2$ é a função externa.

- Então, aplicando a regra da cadeia temos

$$\frac{d}{d\gamma} [(y^{(i)} - \gamma)^2] = 2(y^{(i)} - \gamma) \cdot \frac{d}{d\gamma} (y^{(i)} - \gamma)$$

- Aqui, estamos usando a regra do expoente, que nos dá $2(y^{(i)} - \gamma)$, derivando a função externa. Então, derivamos a função interna, $y^{(i)} - \gamma$, em relação a γ , que resulta em -1 .
- Portanto, temos:

$$\frac{d}{d\gamma} [(y^{(i)} - \gamma)^2] = -2(y^{(i)} - \gamma)$$

Passo 1: Inicializar o modelo

- Substituindo a derivada na expressão original

$$\frac{1}{2} \sum_{i=1}^m \frac{d}{d\gamma} [(y^{(i)} - \gamma)^2] = \frac{1}{2} \sum_{i=1}^m (-2(y^{(i)} - \gamma)) = - \sum_{i=1}^m (y^{(i)} - \gamma)$$

- Igualando a expressão a zero e resolvendo para γ temos

$$- \sum_{i=1}^m (y^{(i)} - \gamma) = 0 \rightarrow - \sum_{i=1}^m y^{(i)} + \sum_{i=1}^m \gamma = 0 \rightarrow - \sum_{i=1}^m y^{(i)} + m\gamma = 0$$

$$m\gamma = \sum_{i=1}^m y^{(i)} \rightarrow \gamma = \frac{1}{m} \sum_{i=1}^m y^{(i)}$$

- Portanto, temos de fato a **média do atributo alvo**

Passo 1: Inicializar o modelo

- Vamos voltar ao nosso exemplo

Área (m ²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000

$$\gamma = \frac{1}{4} \sum_{i=1}^4 y^{(i)} = \frac{1}{4}(1200 + 16500 + 15500 + 14000) = 14500$$

- Neste caso, a primeira versão do modelo é $F_0(x) = 14500$

Passo 1: Inicializar o modelo

- Mas qual a interpretação para $F_0(x) = 14500$?

Isso significa que os valores preditos iniciais, $F_0(x)$, é apenas uma folha, i.e, um modelo muito simples



14500

A folha prevê que todas as instâncias terão o preço 14500

- Agora vamos para passo 2

O passo 2 é extenso, mas vamos fazer isso passo a passo

Passo 2: Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)} \quad \text{para } i = 1, \dots, m$$

- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$
- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(x^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathcal{I}[x \in R_{jt}]$

Passo 2

Passo 2: Para $t = 1$ até T :

- O passo 2 é um loop onde construímos todos os modelos
- De modo geral, vamos construir T modelos, mas na prática, a maioria das pessoas define T como 100 e constrói 100 modelos
- O t minúsculo se refere a um modelo individual. Então, quando t minúsculo é igual a 1, estamos falando sobre o primeiro modelo
- Quando $t=T$, então estamos falando do último modelo

Passo 2.1: Calcular os resíduos

- O próximo passo no GB é calcular os pseudo resíduos r_{ti} para cada instância de treinamento $x^{(i)}$, onde $1 \leq i \leq m$.
- Como estamos no processo de construção do primeiro modelo base, t é igual a 1.
- A Equação para calcular cada pseudo resíduo r_{it} é dada por:

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)}$$

- **O Passo 2.1 parece complicado, mas não é!**

Passo 2.1: Calcular os resíduos

- Lembre-se que iniciamos $t = 1$. Assim $F_{t-1}(X) = F_0(x) = \gamma$, e temos

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, \gamma)}{\partial \gamma} \right]$$

- Mas já calculamos $\frac{\partial \mathcal{L}(y^{(i)}, \gamma)}{\partial \gamma}$

$$\frac{\partial \mathcal{L}(y^{(i)}, \gamma)}{\partial \gamma} = -(y^{(i)} - \gamma)$$

- Portanto

$$r_{i1} = (y^{(i)} - \gamma) = (\mathbf{observado} - \mathbf{predito})$$

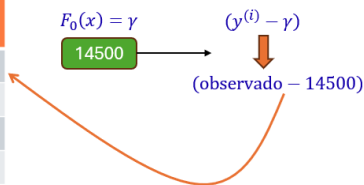
Passo 2.1: Calcular os resíduos

- Assim, calcular os resíduos é simples

$$r_{i1} = (y^{(i)} - \gamma) = (\text{observado} - \text{predito})$$

- Retornando ao nosso exemplo simplificado, temos (considerando que $\gamma = 14500$):

Área (m²)	n_quartos	Região	Preço	r_{i1}
117	3	Sul	12000	-2500
150	3	Norte	16500	2000
140	2	Norte	15500	1000
200	4	Sul	14000	-500



Entendo o conceito de pseudos resíduos

- É importante destacar que os valores r_{it} são tecnicamente chamados de **Pseudo Resíduos**
- Quando usamos a Função de Perda $\Rightarrow \frac{1}{2}(\mathbf{observado} - \mathbf{predito})^2 \Rightarrow$ acabamos calculando Resíduos normais
- Mas se usássemos outra Função de Perda $\Rightarrow (\mathbf{observado} - \mathbf{predito})^2$ então acabaríamos com algo similar a um Resíduo, mas não exatamente igual $\Rightarrow 2(\mathbf{observado} - \mathbf{predito})$
- Em outras palavras, acabaríamos com um Pseudo Resíduo $\Rightarrow 2(\mathbf{observado} - \mathbf{predito})$
- é por isso que os r_{it} s são chamados de Pseudo Resíduos

Passo 2.2: Ajustar um modelo base aos pseudos resíduos

- No próximo passo, o GB constrói um modelo base $h_1(x)$ utilizando os resíduos como alvo

Então nós vamos usar
área, n_quartos e região para prever os Resíduos

Área (m²)	n_quartos	Região	Preço	r_{i1}
117	3	Sul	12000	-2500
150	3	Norte	16500	2000
140	2	Norte	15500	1000
200	4	Sul	14000	-500

- Qual é a razão pela qual o GB constrói um modelo base $h_1(x)$ usando os resíduos como alvo?

Passo 2.2: Ajustar um modelo base aos pseudos resíduos

- Qual é a razão pela qual o GB constrói um modelo base $h_1(x)$ usando os resíduos como alvo?
 - Porque os resíduos codificam os erros que o modelo base anterior comete
 - Se os resíduos forem previstos adequadamente por $h_1(x)$, então podemos adicionar $h_1(x)$ à saída do modelo base anterior:

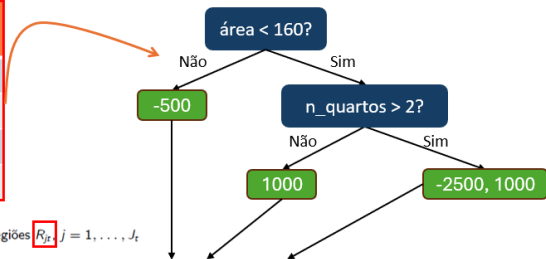
$$F_1(x) = F_0(x) + h_1(x)$$

- Fazendo isso, o modelo resultante (ensemble) F_1 cometerá menos erros

Passo 2.2: Ajustar um modelo base aos pseudos resíduos

- Por exemplo, vamos supor que foi criado a seguinte árvore de decisão como modelo base ajustado aos resíduos $r_{i1}, 1 \leq i \leq 4$

Área (m²)	n_quartos	Região	Preço	r_{i1}
117	3	Sul	12000	-2500
150	3	Norte	16500	2000
140	2	Norte	15500	1000
200	4	Sul	14000	-500



2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$

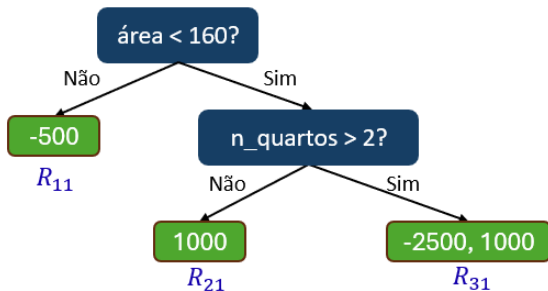
Folhas também são chamadas de regiões terminais R_{jt}

Nota: Esse t minúsculo é o índice para a árvore que acabamos de criar e o j minúsculo é o índice para cada folha na árvore

Passo 2.2: Ajustar um modelo base aos pseudos resíduos

Uma vez que essa árvore tem
3 folhas, $J_t = 3$

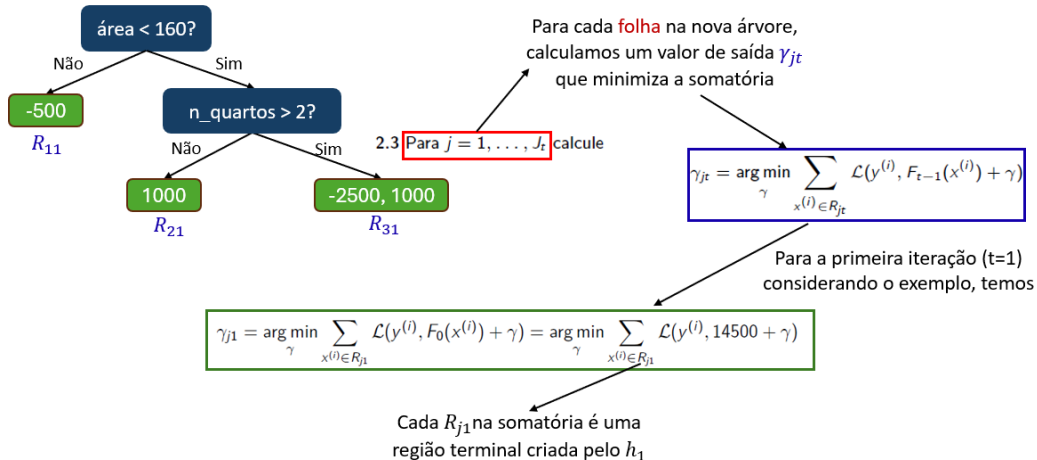
2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões $R_{jt}, j = 1, \dots, J_t$



- $R_{11} \rightarrow$ região na qual a área ≥ 160
- $R_{21} \rightarrow$ região na qual a área < 160 e n_quartos ≤ 2
- $R_{31} \rightarrow$ região na qual a área < 160 e n_quartos > 2

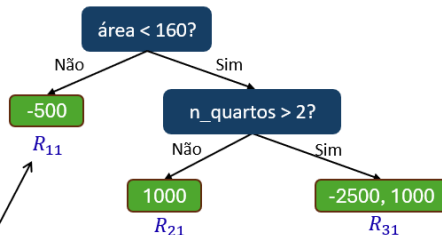
Essas regiões terminais entram em cena no próximo passo do GB

Passo 2.3: Calcular os valores de γ



Passo 2.3: Calcular os valores de γ

Área (m²)	n_quartos	Região	Preço	r_{i1}
117	3	Sul	12000	-2500
150	3	Norte	16500	2000
140	2	Norte	15500	1000
200	4	Sul	14000	-500

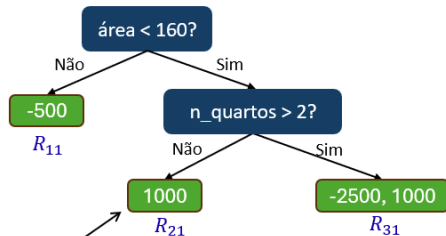


Somente a instância $x^{(4)}$ é usada para calcular o valor de saída para R_{11}

$$\gamma_{j1} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{j1}} \mathcal{L}(y^{(i)}, 14500 + \gamma)$$

Passo 2.3: Calcular os valores de γ

Área (m²)	n_quartos	Região	Preço	r_{i1}
117	3	Sul	12000	-2500
150	3	Norte	16500	2000
140	2	Norte	15500	1000
200	4	Sul	14000	-500

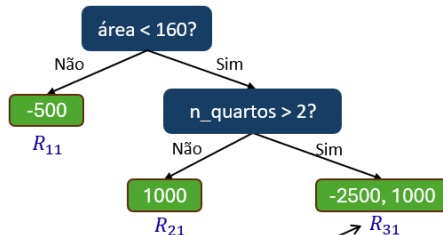


$$\gamma_{j1} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{j1}} \mathcal{L}(y^{(i)}, 14500 + \gamma)$$

Também somente a instância $x^{(3)}$ é usada para calcular o valor de saída para R_{21}

Passo 2.3: Calcular os valores de γ

Área (m²)	n_quartos	Região	Preço	r_{i1}
117	3	Sul	12000	-2500
150	3	Norte	16500	2000
140	2	Norte	15500	1000
200	4	Sul	14000	-500



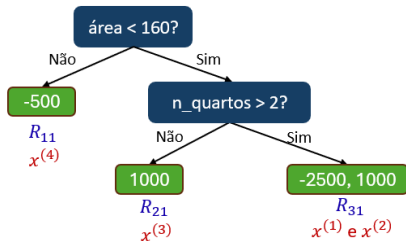
As instâncias $x^{(1)}$ e $x^{(2)}$ são usadas para calcular o valor de saída para R_{31}

$$\gamma_{j1} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{j1}} \mathcal{L}(y^{(i)}, 14500 + \gamma)$$

Passo 2.3: Calcular os valores de γ

- Em resumo cada instância pertence as seguintes regiões

	Área (m²)	n_quartos	Região	Preço	r_{i1}
$x^{(1)}$	117	3	Sul	12000	-2500
$x^{(2)}$	150	3	Norte	16500	2000
$x^{(3)}$	140	2	Norte	15500	1000
$x^{(4)}$	200	4	Sul	14000	-500



- Assim, podemos calcular cada valor de γ

Passo 2.3: Calcular os valores de γ

- Calculando γ_{11}

$$\gamma_{11} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{11}} \mathcal{L}(y^{(i)}, 14500 + \gamma)$$

$$= \arg \min_{\gamma} \mathcal{L}(y^{(4)}, 14500 + \gamma)$$

$$= \arg \min_{\gamma} \mathcal{L}(14000, 14500 + \gamma)$$

$$= \arg \min_{\gamma} 1/2(14000 - (14500 + \gamma))^2$$

$$\gamma_{11} = \arg \min_{\gamma} 1/2(-500 - \gamma)^2$$

- Precisamos encontrar o valor de γ que minimiza a função

- Podemos resolver analiticamente aplicando a derivada da função perda

$$\frac{d}{d\gamma} \frac{1}{2}(-500 - \gamma)^2$$

- Aplicando a Regra da cadeia e depois igualando a zero temos

$$(-500 - \gamma) * \frac{d}{d\gamma}(-500 - \gamma)$$

$$= (-500 - \gamma) * (-1) = 500 + \gamma$$

$$500 + \gamma = 0 \rightarrow \gamma = -500$$

- Logo $\gamma_{11} = -500$

Passo 2.3: Calcular os valores de γ

- Calculando γ_{21}

$$\begin{aligned}\gamma_{21} &= \arg \min_{\gamma} \sum_{\mathbf{x}^{(i)} \in R_{21}} \mathcal{L}(y^{(i)}, 14500 + \gamma) \\ &= \arg \min_{\gamma} \mathcal{L}(y^{(3)}, 14500 + \gamma) \\ &= \arg \min_{\gamma} \mathcal{L}(15500, 14500 + \gamma) \\ &= \arg \min_{\gamma} 1/2(15500 - (14500 + \gamma))^2 \\ &= \arg \min_{\gamma} 1/2(1000 - \gamma)^2\end{aligned}$$

- A solução para encontrar o valor de γ é a mesma da anterior. Logo $\gamma_{21} = 1000$

Passo 2.3: Calcular os valores de γ

- Calculando γ_{31}

$$\begin{aligned}\gamma_{31} &= \arg \min_{\gamma} \sum_{x^{(i)} \in R_{31}} \mathcal{L}(y^{(i)}, 14500 + \gamma) \\&= \arg \min_{\gamma} [\mathcal{L}(y^{(1)}, 14500 + \gamma) + \mathcal{L}(y^{(2)}, 14500 + \gamma)] \\&= \arg \min_{\gamma} [\mathcal{L}(12000, 14500 + \gamma) + \mathcal{L}(16500, 14500 + \gamma)] \\&= \arg \min_{\gamma} [1/2(12000 - (14500 + \gamma))^2 + 1/2(16500 - (14500 + \gamma))^2] \\&= \arg \min_{\gamma} [1/2(-2500 - \gamma))^2 + 1/2(2000 - \gamma))^2]\end{aligned}$$

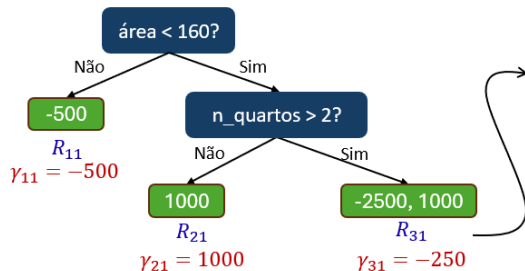
- Calculando a derivada aplicando a regra da cadeia e depois igualando a zero

$$\frac{d}{d\gamma} \left[\frac{1}{2}(-2500 - \gamma))^2 + \frac{1}{2}(2000 - \gamma))^2 \right] \rightarrow [(-2500 - \gamma) * (-1)] + [(2000 - \gamma) * (-1)]$$

$$2500 + \gamma - 2000 + \gamma = 0 \rightarrow \gamma = \frac{2000 - 2500}{2} = -250$$

- Assim $\gamma_{31} = -250$

Passo 2.3: Calcular os valores de γ



- Acabamos de ver que o valor de saída para esta folha R_{31} é a média dos resíduos que acabaram aqui

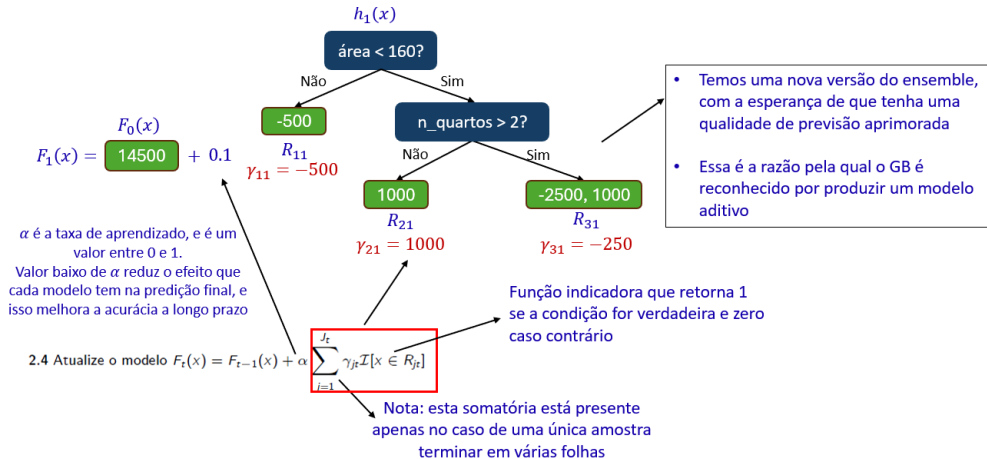
- Dada de Função de Perda escolhida, os valores de saída são sempre a média dos resíduos que terminam na mesma folha

- Por exemplo em R_{31} temos
$$\frac{-2500+1000}{2} = -250$$
 ou mesmo em
$$R_{11} \rightarrow \frac{-500}{1} = -500$$

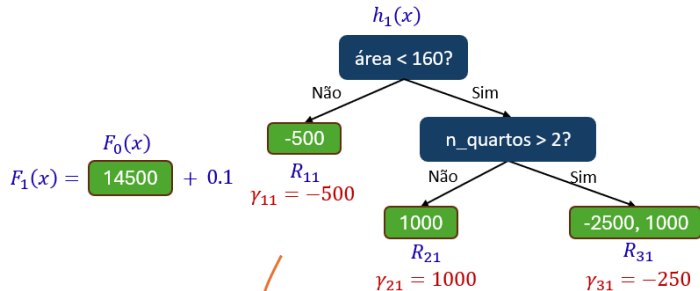
$$\frac{1}{2} (\text{observado} - \text{predito})^2$$

Passo 2.4: Atualize o modelo

- Neste último passo, o GB incorpora o novo modelo base (h_t) na versão atual (f_{t-1}) do modelo ensemble para produzir sua nova versão (f_t)



Passo 2.4: Atualize o modelo

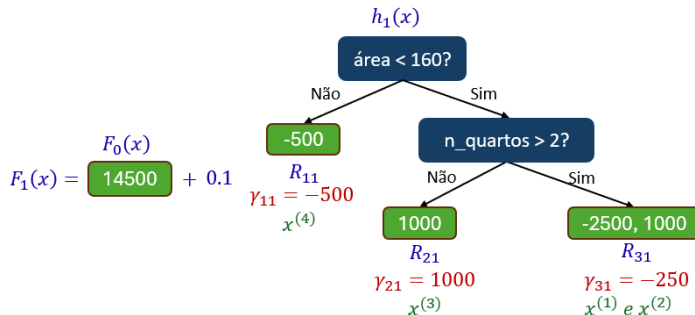


Área (m²)	n_quartos	Região	Preço
117	3	Sul	12000
150	3	Norte	16500
140	2	Norte	15500
200	4	Sul	14000

Vamos usar $F_1(x)$ para fazer as novas previsões para cada instância

2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathcal{I}[x \in R_{jt}]$

Passo 2.4: Atualize o modelo

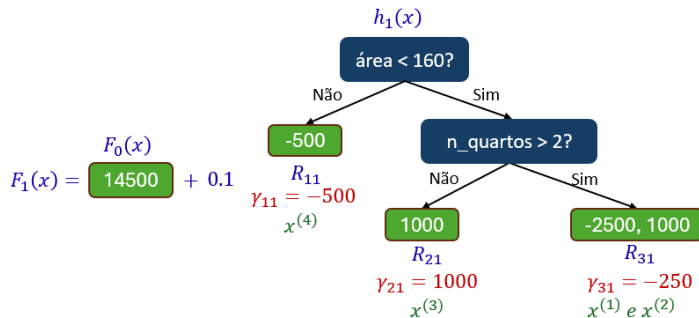


	Área (m²)	n_quartos	Região	Preço	h_1	αh_1	F_0	F_1
$x^{(1)}$	117	3	Sul	12000	-250	-25.0	14500	14475
$x^{(2)}$	150	3	Norte	16500			14500	
$x^{(3)}$	140	2	Norte	15500			14500	
$x^{(4)}$	200	4	Sul	14000			14500	

$$F_1(x) = F_0(x) + \alpha \cdot h_1(x)$$

A nova predição para $x^{(1)}$ é
 $F_1 = 14500 + 0.1 \cdot (-250) = 14475$

Passo 2.4: Atualize o modelo

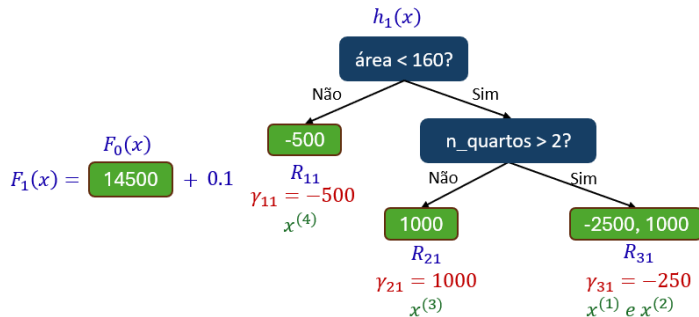


	Área (m²)	n_quartos	Região	Preço	h_1	αh_1	F_0	F_1
$x^{(1)}$	117	3	Sul	12000	-250	-25.0	14500	14475
$x^{(2)}$	150	3	Norte	16500	-250	-25.0	14500	14475
$x^{(3)}$	140	2	Norte	15500			14500	
$x^{(4)}$	200	4	Sul	14000			14500	

$$F_1(x) = F_0(x) + \alpha \cdot h_1(x)$$

A nova predição para $x^{(2)}$ é
 $F_1 = 14500 + 0.1 \cdot (-250) = 14475$

Passo 2.4: Atualize o modelo

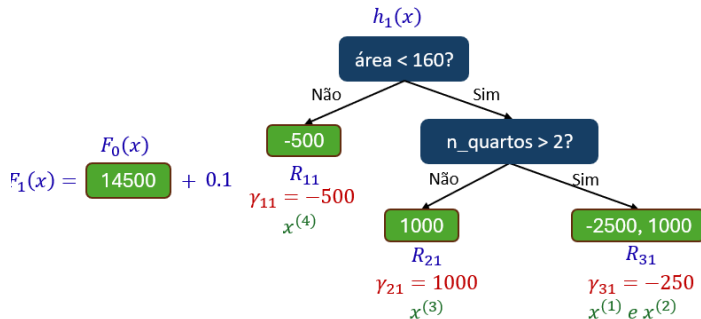


	Área (m²)	n_quartos	Região	Preço	h_1	αh_1	F_0	F_1
$x^{(1)}$	117	3	Sul	12000	-250	-25.0	14500	14475
$x^{(2)}$	150	3	Norte	16500	-250	-25.0	14500	14475
$x^{(3)}$	140	2	Norte	15500	1000	100.0	14500	14600
$x^{(4)}$	200	4	Sul	14000			14500	

$$F_1(x) = F_0(x) + \alpha \cdot h_1(x)$$

A nova predição para $x^{(3)}$ é
 $F_1 = 14500 + 0.1 \cdot (1000) = 14600$

Passo 2.4: Atualize o modelo



	Área (m²)	n_quartos	Região	Preço	h_1	αh_1	F_0	F_1
$x^{(1)}$	117	3	Sul	12000	-250	-25.0	14500	14475
$x^{(2)}$	150	3	Norte	16500	-250	-25.0	14500	14475
$x^{(3)}$	140	2	Norte	15500	1000	100.0	14500	14600
$x^{(4)}$	200	4	Sul	14000	-500	-50.0	14500	14450

$$F_1(x) = F_0(x) + \alpha \cdot h_1(x)$$

A nova predição para $x^{(4)}$ é
 $F_1 = 14500 + 0.1 \cdot (-500) = 14450$

Conclusão da iteração 1 do passo 2

- Concluimos a primeira iteração do **passo 2**

Passo 2: Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)} \quad \text{para } i = 1, \dots, m$$

- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$
- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(x^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathbb{I}[x \in R_{jt}]$

- Considere $T = 2$ e então vamos para próxima e última iteração $t = 2$

Iteração 2 do Passo 2

- Calcule novos resíduos para as novas previsões

Passo 2: Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)} \quad \text{para } i = 1, \dots, m$$

- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$
- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(x^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathcal{I}[x \in R_{jt}]$

Área (m²)	n_quartos	Região	Preço	F_1	r_{i2}
117	3	Sul	12000	14475	-2475
150	3	Norte	16500	14475	2025
140	2	Norte	15500	14600	900
200	4	Sul	14000	14450	-450

Iteração 2 do Passo 2

- Cria um novo modelo $h_2(x)$

Passo 2: Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)} \quad \text{para } i = 1, \dots, m$$

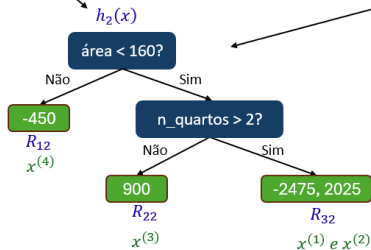
- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$

- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(x^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathcal{I}[x \in R_{jt}]$

Área (m²)	n_quartos	Região	Preço	F_1	r_{i2}
117	3	Sul	12000	14475	-2475
150	3	Norte	16500	14475	2025
140	2	Norte	15500	14600	900
200	4	Sul	14000	14450	-450



Iteração 2 do Passo 2

- Calculando os valores de saída γ

Passo 2: Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)} \quad \text{para } i = 1, \dots, m$$

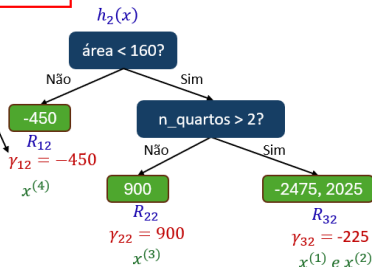
- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$

- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(x^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathbb{I}[x \in R_{jt}]$

Área (m²)	n_quartos	Região	Preço	F_1	r_{i2}
117	3	Sul	12000	14475	-2475
150	3	Norte	16500	14475	2025
140	2	Norte	15500	14600	900
200	4	Sul	14000	14450	-450



Iteração 2 do Passo 2

● Atualizar o modelo $F_2(x)$

Passo 2: Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)} \quad \text{para } i = 1, \dots, m$$

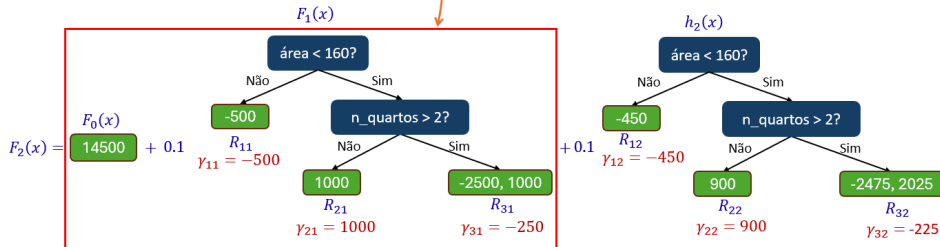
- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$
- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(x^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathbb{I}[x \in R_{jt}]$

Na segunda iteração do passo 2, e as novas previsões, $F_2(x)$, são baseadas nas previsões feitas por $F_1(x)$

Área (m²)	n_quartos	Região	Preço	F_1	r_{i2}
117	3	Sul	12000	14475	-2475
150	3	Norte	16500	14475	2025
140	2	Norte	15500	14600	900
200	4	Sul	14000	14450	-450



Iteração 2 do Passo 2

- Por questão de tempo vamos considerar $T = 2$ e então paramos nessa iteração 2
- Lembrando que na prática o valor de $T = 100$ ou mais

Passo 2: Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F(x)=F_{t-1}(x)} \quad \text{para } i = 1, \dots, m$$

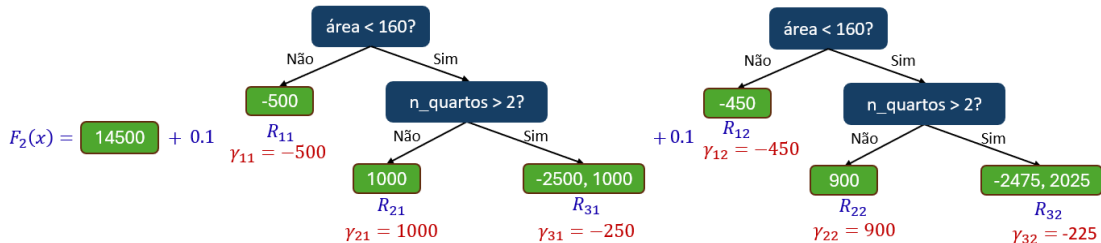
- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$
- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{x^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(x^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(x) = F_{t-1}(x) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathcal{I}[x \in R_{jt}]$

Passo 3: Retorne o modelo final

- No terceiro e último passo o algoritmo GB retorna o modelo final
- No caso como $T = 2$, o algoritmo GB retorna $F_2(x)$

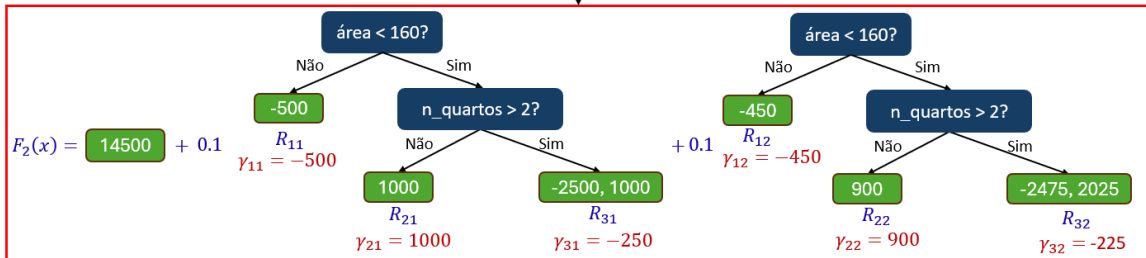


Saída do GB para uma nova entrada

Considerando uma nova instância

Área (m²)	n_quartos	Região	Preço
167	5	Sul	???

Usamos $F_2(x)$ para prever o preço do imóvel



$$\text{Preço predito} = 14500 + 0.1 * (-500) + 0.1 * (-450) = 14405$$

- Slides de:
 - Prof. Dr. Eduardo Bezerra, CFET-RJ
 - Josh Starmer, StatQuest

- Algoritmo do Gradiente Boosting em Classificação

Algoritmo do Gradiente Boosting em Classificação

Jefferson Moraes

Gestorlearn

Algoritmo do Gradiente Boosting

- **Entrada:** Conjunto de Dados $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, onde $y^{(i)} \in \mathbb{R}$ e uma função de perda (*loss function*) diferenciável $\mathcal{L}(y^{(i)}, F(\mathbf{x}))$

- **Passo 1:** Inicialize o modelo com um valor constante: $F_0(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^m \mathcal{L}(y^{(i)}, \gamma)$

- **Passo 2:** Para $t = 1$ até T :

- 2.1 Compute os erros residuais r_{it} :

$$r_{it} = - \left[\frac{\partial \mathcal{L}(y^{(i)}, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})} \right]_{F(\mathbf{x})=F_{t-1}(\mathbf{x})} \quad \text{para } i = 1, \dots, m$$

- 2.2 Ajuste o modelo base $h_t(x)$ aos pseudos resíduos r_{it} , que produz regiões R_{jt} , $j = 1, \dots, J_t$

- 2.3 Para $j = 1, \dots, J_t$ calcule

$$\gamma_{jt} = \arg \min_{\gamma} \sum_{\mathbf{x}^{(i)} \in R_{jt}} \mathcal{L}(y^{(i)}, F_{t-1}(\mathbf{x}^{(i)}) + \gamma)$$

- 2.4 Atualize o modelo $F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \alpha \sum_{j=1}^{J_t} \gamma_{jt} \mathcal{I}[\mathbf{x} \in R_{jt}]$

- Retorne $F_T(\mathbf{x})$

GB em classificação

- Objetivo é prever se uma pessoa assiste ou não à Netflix com base em sua preferência por gatos, idade e esporte favorito
- Problema de classificação binária

Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix
Sim	12	Basquete	Sim
Sim	87	Futebol	Sim
Não	44	Basquete	Não
Sim	19	Vôlei	Não
Não	32	Futebol	Sim
Não	14	Basquete	Sim

- Agora vamos ver como o algoritmo GB resolve esse problema

- Começamos com **um nó folha** que prevê o valor inicial para cada indivíduo
- Para um problema de classificação, isso seria o $\log(\text{odds})$ inicial de uma pessoa assistir à Netflix
 - O $\log(\text{odds})$ é o logaritmo da razão entre a probabilidade de pertencer à classe positiva (ex: assistir à Netflix) e a probabilidade de pertencer à classe negativa (ex: não assistir à Netflix)
 - Em outras palavras, o $\log(\text{odds})$ representa a “razão de chances” entre as duas classes
- Como 4 pessoas assistiram à Netflix e duas não assistiram, a probabilidade de uma pessoa assistir à Netflix seria:

$$\log\left(\frac{\text{assisti}}{\text{não assisti}}\right) = \log\left(\frac{4}{2}\right) = 0.693 \approx 0.7$$

$$\log(4/2) = 0.7$$

← Esta é a **predição inicial**

- Assim como com a **Regressão Logística**, a maneira mais fácil de usar o $\log(odds)$ para classificação é convertê-lo em uma **probabilidade**
- Fazemos isso usando a **Função logística**

$$P(\text{assistir}) = \frac{e^{\log(odds)}}{1 + e^{\log(odds)}}$$

- Substituindo os valores temos

$$P(\text{assistir}) = \frac{e^{\log(4/2)}}{1 + e^{\log(4/2)}} = 0.667 \approx 0.7$$

$$\log(4/2) = 0.7$$

*Probabilidade
de Assistir Netflix = 0.7*

Se a probabilidade de assistir Netflix for superior a **0.5**, primeiro classificamos todos no **conjunto de dados de treinamento** como alguém que assistiu Netflix

Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix
Sim	12	Basquete	Sim
Sim	87	Futebol	Sim
Não	44	Basquete	Não
Sim	19	Vôlei	Não
Não	32	Futebol	Sim
Não	14	Basquete	Sim

Nota: Embora 0.5 seja um limiar muito comum para tomar decisões de classificação com base em probabilidades, poderíamos ter usado facilmente um valor diferente.

GB em Classificação

$$\log(4/2) = 0.7$$

*Probabilidade
de Assistir Netflix = 0.7*



Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix
Sim	12	Basquete	Sim
Sim	87	Futebol	Sim
Não	44	Basquete	Não
Sim	19	Vôlei	Não
Não	32	Futebol	Sim
Não	14	Basquete	Sim

Podemos medir quão ruim é a predição inicial calculando os pseudo-resíduos, diferença entre os valores observados e os valores preditos

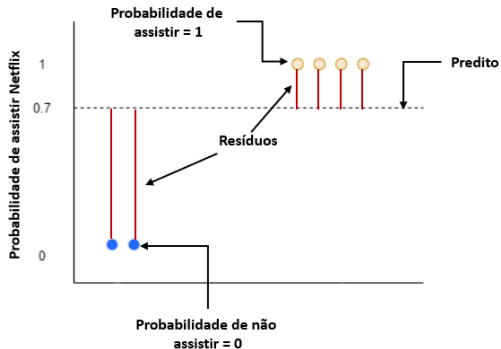
$$\text{Resíduo} = \text{Observado} - \text{Predito}$$

GB em Classificação

$$\log(4/2) = 0.7$$

*Probabilidade
de Assistir Netflix = 0.7*

Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix
Sim	12	Basquete	Sim
Sim	87	Futebol	Sim
Não	44	Basquete	Não
Sim	19	Vôlei	Não
Não	32	Futebol	Sim
Não	14	Basquete	Sim



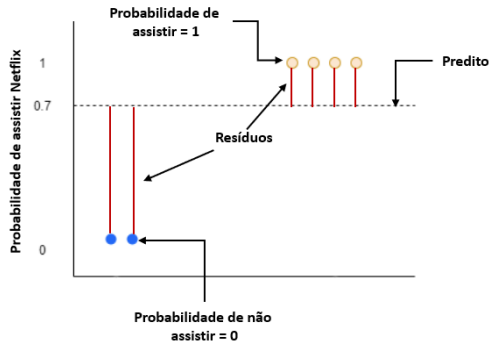
GB em Classificação

$$\log(4/2) = 0.7$$

Probabilidade
de Assistir Netflix = 0.7



Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix	Resíduo
Sim	12	Basquete	1	0.3
Sim	87	Futebol	1	0.3
Não	44	Basquete	0	-0.7
Sim	19	Vôlei	0	-0.7
Não	32	Futebol	1	0.3
Não	14	Basquete	1	0.3



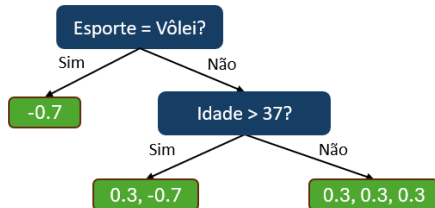
- Usaremos esse resíduo para obter a próxima árvore

GB em Classificação

$$\log(4/2) = 0.7$$

*Probabilidade
de Assistir Netflix = 0.7*

Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix	Resíduo
Sim	12	Basquete	1	0.3
Sim	87	Futebol	1	0.3
Não	44	Basquete	0	-0.7
Sim	19	Vôlei	0	-0.7
Não	32	Futebol	1	0.3
Não	14	Basquete	1	0.3



- Assim como no GB para Regressão, estamos limitando o número de folhas que permitiremos na árvore
- Neste exemplo, limitamos o número de folhas para 3
- Na prática se usa o número máximo de folhas entre 8 e 32

- Uma única folha pode abranger **várias observações**
- As previsões são expressas em termos de log(odds)
- Enquanto as folhas são derivadas de probabilidades, o que pode causar **discrepâncias**
- É necessário aplicar uma **transformação para harmonizar** esses diferentes aspectos.
- Alternativa comum de transformação utilizada é

$$\frac{\sum \text{Resíduos}}{\sum [\text{ProbPrevia} * (1 - \text{ProbPrevia})]}$$

- O numerador nesta equação é a soma dos resíduos naquela folha específica
- O denominador é a soma de (probabilidade de predição anterior para cada resíduo) * (1 - mesma probabilidade de predição anterior)

GB em Classificação

$$\log(4/2) = 0.7$$

*Probabilidade
de Assistir Netflix = 0.7*

$$\frac{\sum \text{Resíduos}}{\sum [\text{ProbPrevia} * (1 - \text{ProbPrevia})]}$$

$$= \frac{-0.7}{0.7 * (1 - 0.7)}$$

Novo valor de saída

-0.7

-3.3

Esporte = Vôlei?

Sim

Não

Idade > 37?

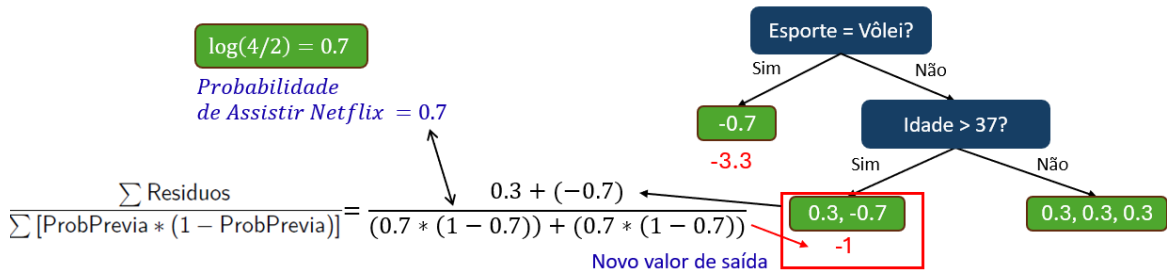
Sim

Não

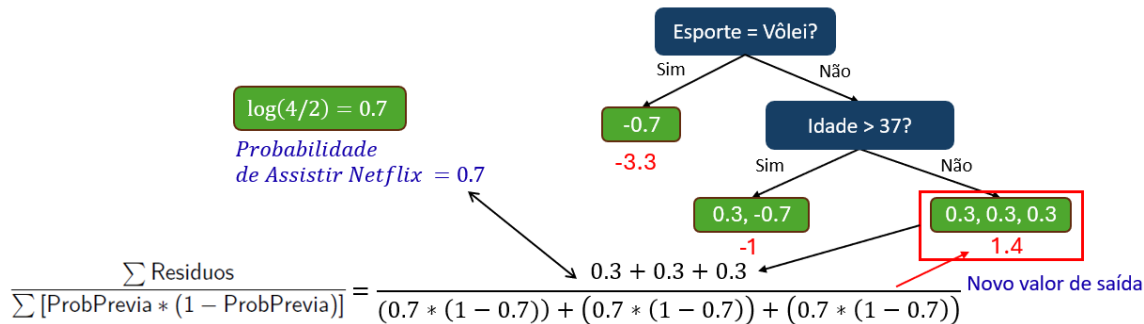
0.3, -0.7

0.3, 0.3, 0.3

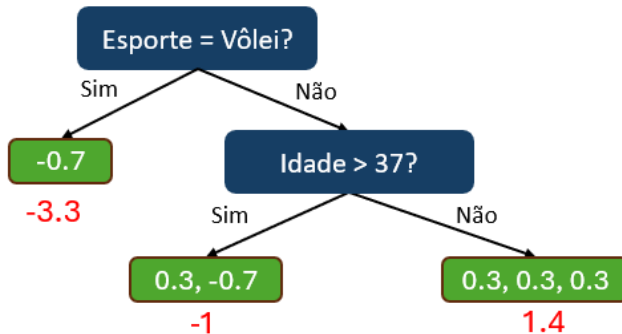
GB em Classificação



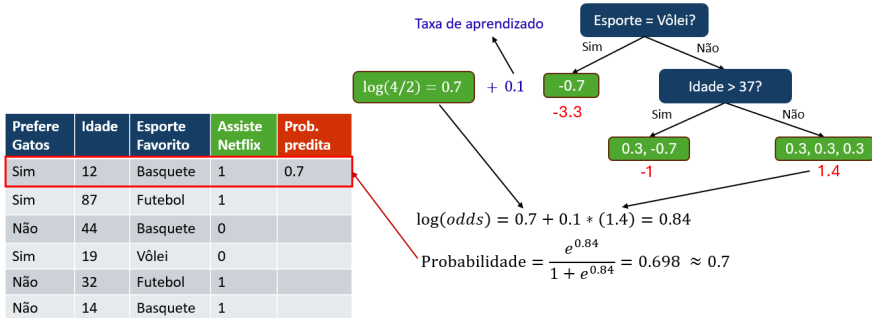
GB em Classificação



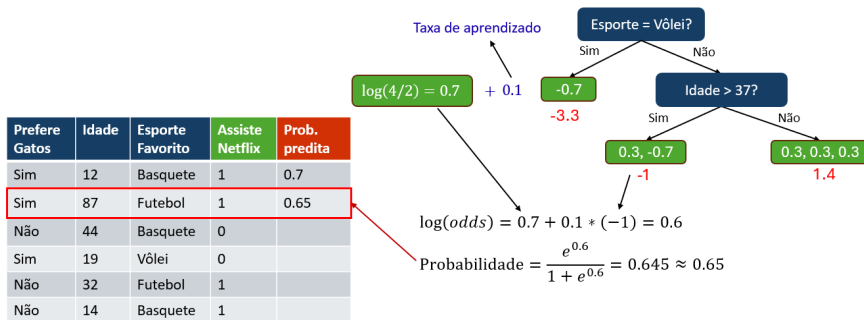
- Árvore transformada



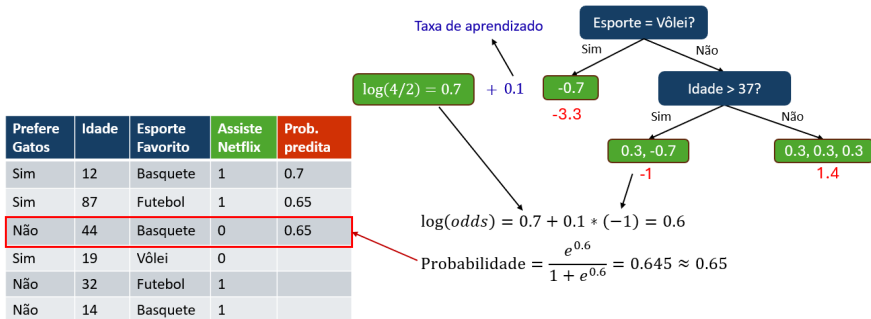
- Calculando as novas probabilidades



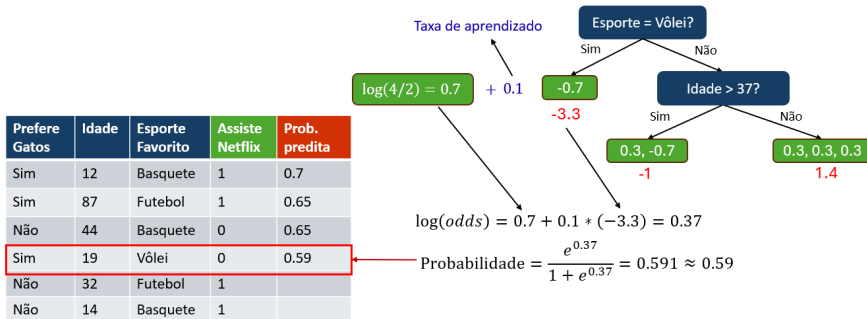
- Calculando as novas probabilidades



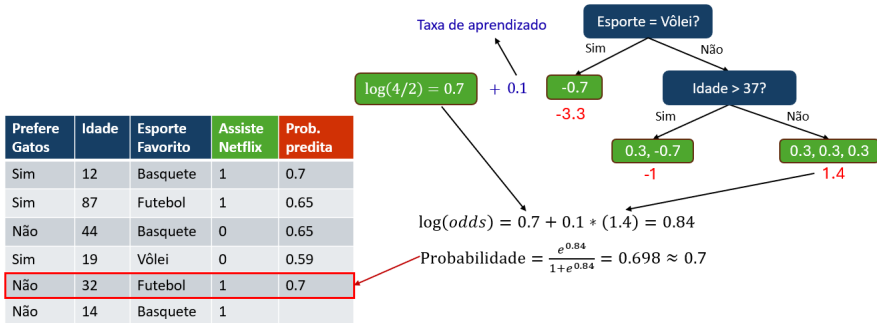
- Calculando as novas probabilidades



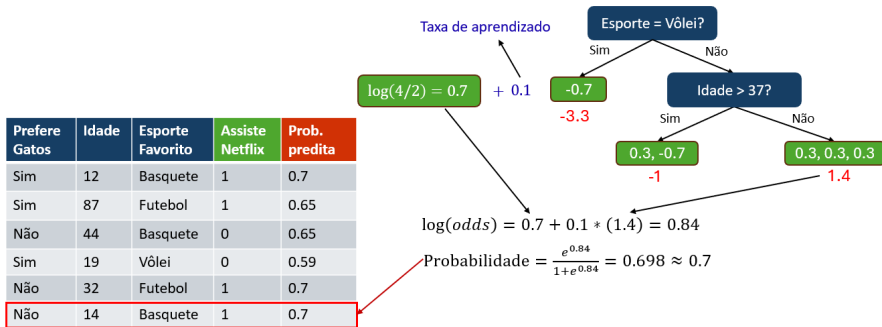
- Calculando as novas probabilidades



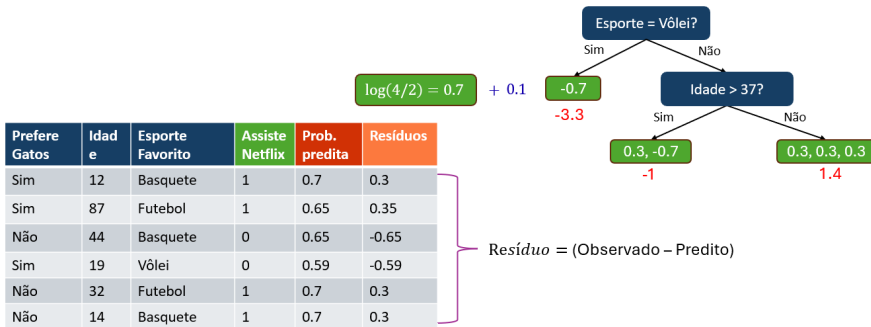
- Calculando as novas probabilidades



- Calculando as novas probabilidades

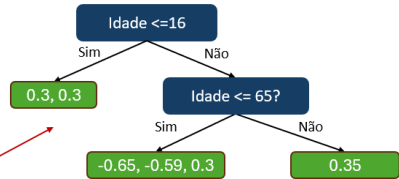


- Calculando agora os novos resíduos



- Nova árvore construída

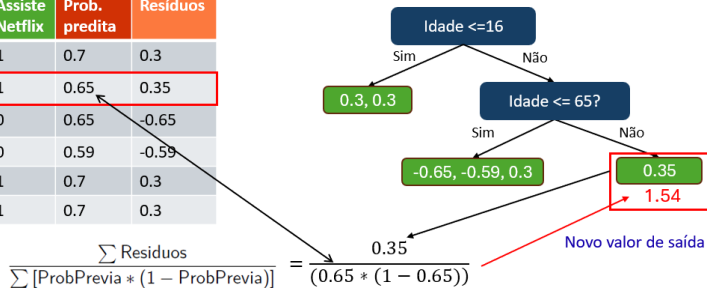
Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix	Prob. predita	Resíduos
Sim	12	Basquete	1	0.7	0.3
Sim	87	Futebol	1	0.65	0.35
Não	44	Basquete	0	0.65	-0.65
Sim	19	Vôlei	0	0.59	-0.59
Não	32	Futebol	1	0.7	0.3
Não	14	Basquete	1	0.7	0.3



- Agora vamos calcular as novas saídas

- Calculando as novas saídas para cada folha

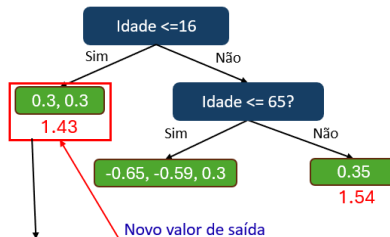
Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix	Prob. predita	Resíduos
Sim	12	Basquete	1	0.7	0.3
Sim	87	Futebol	1	0.65	0.35
Não	44	Basquete	0	0.65	-0.65
Sim	19	Vôlei	0	0.59	-0.59
Não	32	Futebol	1	0.7	0.3
Não	14	Basquete	1	0.7	0.3



- Calculando as novas saídas para cada folha

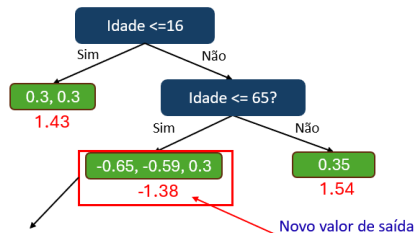
Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix	Prob. predita	Resíduos
Sim	12	Basquete	1	0.7	0.3
Sim	87	Futebol	1	0.65	0.35
Não	44	Basquete	0	0.65	-0.65
Sim	19	Vôlei	0	0.59	-0.59
Não	32	Futebol	1	0.7	0.3
Não	14	Basquete	1	0.7	0.3

$$\frac{\sum \text{Resíduos}}{\sum [\text{ProbPrev} * (1 - \text{ProbPrev})]} = \frac{0.3 + 0.3}{(0.7 * (1 - 0.7)) + (0.7 * (1 - 0.7))}$$



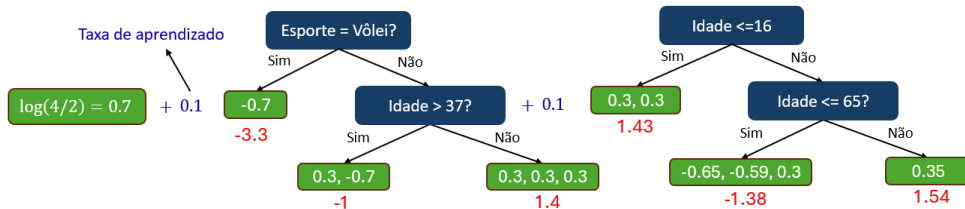
- Calculando as novas saídas para cada folha

Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix	Prob. predita	Resíduos
Sim	12	Basquete	1	0.7	0.3
Sim	87	Futebol	1	0.65	0.35
Não	44	Basquete	0	0.65	-0.65
Sim	19	Vôlei	0	0.59	-0.59
Não	32	Futebol	1	0.7	0.3
Não	14	Basquete	1	0.7	0.3



$$\frac{\sum \text{Resíduos}}{\sum [\text{ProbPrev} * (1 - \text{ProbPrev})]} = \frac{-0.65 - 0.59 + 0.3}{(0.65 * (1 - 0.65)) + (0.59 * (1 - 0.59)) + (0.7 * (1 - 0.7))}$$

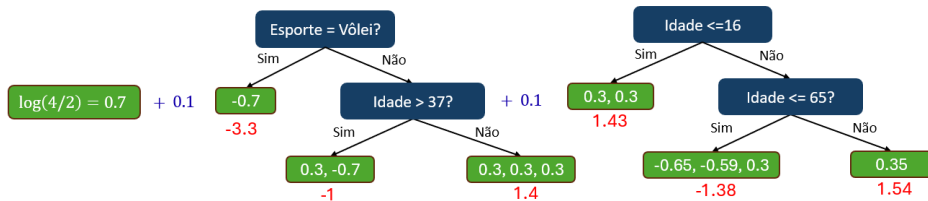
- Novo modelo criado



- Observação: O processo repete-se até o número máximo de modelos (árvores neste caso) especificado
- Para efeito de simplificação vamos considerar que esse é o nosso modelo final

- Vamos verificar se a pessoa abaixo **assisti ou não assisti Netflix**

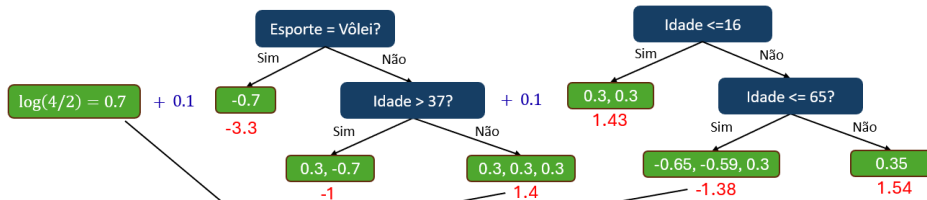
Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix
Sim	25	Basquete	???



GB em Classificação

- Vamos verificar se a pessoa abaixo **assisti ou não assisti Netflix**

Prefere Gatos	Idade	Esporte Favorito	Assiste Netflix
Sim	25	Basquete	???



$$\log(odds) = 0.7 + 0.1 * (1.4) + 0.1 * (-1.38) \approx 0.7$$

$$\text{Probabilidade} = \frac{e^{0.7}}{1 + e^{0.7}} = 0.668 \approx 0.67$$

- Probabilidade da pessoa assistir Netflix
- Como a probabilidade é maior que 0.5, logo a nova instância será classificada como 1 (sim), ou seja, **Assisti Netflix**

- Implementação do Gradiente Boosting

Ensemble Learning - Stacking

Jefferson Morais

Gestorlearn

- Stacking, abreviação para *stacked generalization*¹, é uma técnica de *Ensemble Learning* que visa combinar múltiplos modelos para criar uma predição mais robusta e precisa
- **Ideia simples:** em vez de usar funções triviais (como votação majoritária) para agregar as predições de todos os modelos em um ensemble, por que não treinar um modelo para realizar essa agregação?
- Assim, diferentes modelos (chamados de modelos de base) são treinados separadamente
- Em seguida, as predições desses modelos de base são usadas como entradas para um segundo nível de modelo (meta-modelo)
- Este meta-modelo aprende a melhor forma de combinar as predições dos modelos de base

¹David H. Wolpert, **Stacked Generalization**, Neural Networks 5, no. 2 (1992): 241–259.

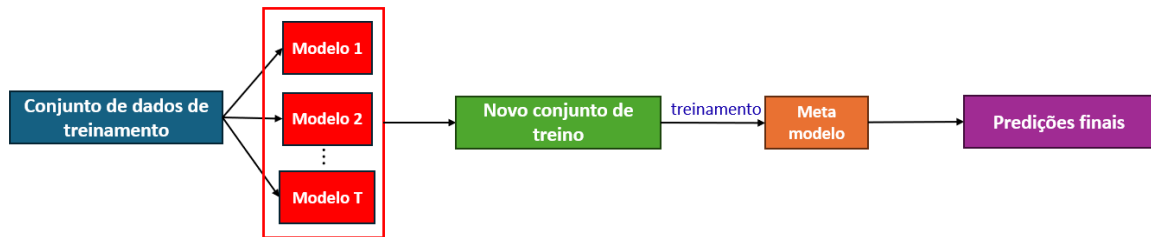
- Diferenças em relação ao **Bagging**

- No stacking, os modelos geralmente são diferentes (por exemplo, não são todos árvores de decisão)
- Os modelos são ajustados no mesmo conjunto de dados, em vez de amostras do conjunto de dados de treinamento

- Diferenças em relação ao **Boosting**

- No stacking, um único modelo é usado para aprender como combinar da melhor forma as previsões dos modelos contribuidores
- Em vez de uma sequência de modelos que corrigem as previsões dos modelos anteriores, há um único modelo combinador

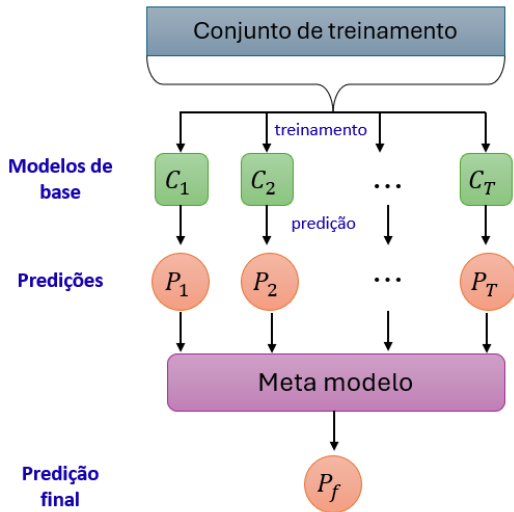
Arquitetura do Stacking



Passos Gerais para construir um Stacking

- 1 Treine os modelos de primeiro nível h_t (onde $1 \leq t \leq T$) usando o conjunto de dados original $\mathcal{D}$
- 2 Crie um novo conjunto de dados $\mathcal{D}'$ baseado nas previsões feitas pelos T modelos de primeiro nível
- 3 Treine um modelo de segundo nível (h' , meta-modelo) no novo conjunto de dados $\mathcal{D}'$
- Existem duas abordagens para implementar o stacking: **sem validação cruzada** e **com validação cruzada**

Stacking sem validação cruzada



Stacking sem validação cruzada

Algorithm 1: Stacking

Entrada: Conjunto de treinamento $\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^m$

Saida : Um comitê de modelos de base H

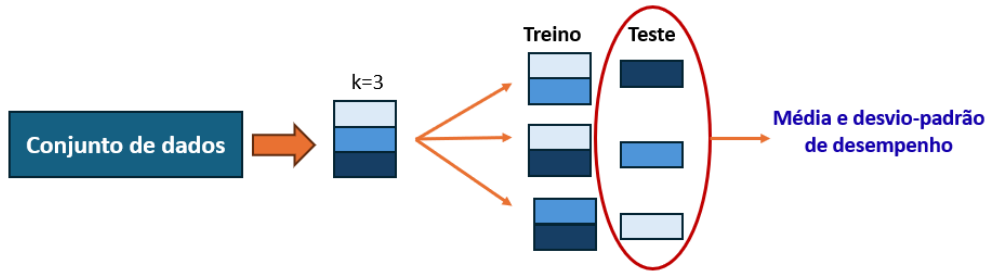
- 1: **Passo 1:** Treino dos classificadores do primeiro nível
 - 2: **for** $t = 1$ to T **do**
 - 3: Treine um modelo de base h_t em relação a $\mathcal{D}$
 - 4: **end for**
 - 5: **Passo 2:** Construa novos conjuntos de dados a partir de $\mathcal{D}$
 - 6: **for** $i = 1$ to m **do**
 - 7: Construa um novo conjunto de dados $\mathcal{D}' = \{\mathbf{x}'^{(i)}, y^{(i)}\}$, onde
 $\mathbf{x}'^{(i)} = \{h_1(\mathbf{x}^{(i)}), h_2(\mathbf{x}^{(i)}), \dots, h_T(\mathbf{x}^{(i)})\}$
 - 8: **end for**
 - 9: **Passo 3:** Treine o modelo de segundo nível (meta-modelo) em $\mathcal{D}'$
 - 10: **return** $H(\mathbf{x}) = h'(\mathbf{x}) = h'(h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_T(\mathbf{x}))$
-

Stacking sem validação cruzada

- Nesta versão do Stacking, há um risco de que o modelo combinado sofra de **overfitting**
 - Isso acontece porque cada h_t faz previsões nos mesmos dados que foram usados para treiná-lo
- Para evitar o overfitting, podemos usar a **validação cruzada** na implementação do stacking
 - Objetivo: obter uma estimativa imparcial das previsões do modelo combinado em dados não vistos
 - Os exemplos em $\mathcal{D}'$ são gerados apenas a partir de exemplos em $\mathcal{D}$ que não foram utilizados para treinar os modelos de primeiro nível
- Mas como funciona a validação cruzada?

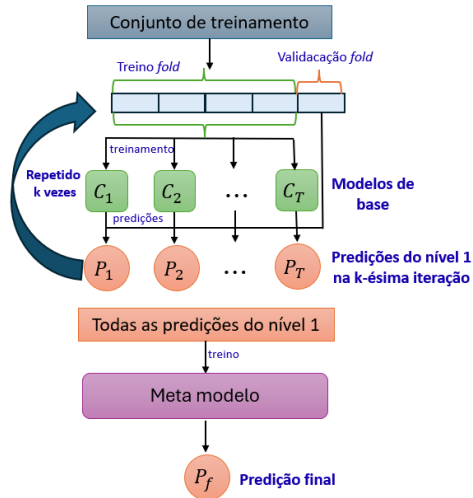
- Método mais usado: Validação cruzada *k-fold*
 - Conjunto é dividido em k partes de tamanho aproximadamente igual
 - Exemplos de $k - 1$ partes são usados no treinamento e a parte restante é usada para teste
 - Procedimento é repetido k vezes usando cada partição para teste
 - $\Rightarrow$ subconjuntos de teste são independentes entre si
 - Desempenho é dado pela média das execuções e o valor típico de $k = 10$

Validação Cruzada



Stacking com validação cruzada

- O conjunto de dados é dividido em **k grupos** (folds)
- Em **k rodadas sucessivas**, **k-1 grupos** são usados para treinar o classificador de primeiro nível.
- Em cada rodada, o classificador de primeiro nível é aplicado ao **único grupo restante** que não foi usado para treinar o modelo naquela iteração.
- As previsões resultantes são então **empilhadas** e fornecidas como **dados de entrada** para o classificador de segundo nível.
- Após o treinamento do classificador stacking, os classificadores de primeiro nível são ajustados usando todo o conjunto de dados



Stacking com validação cruzada

Algorithm 2: Stacking com Validação cruzada k -fold

Entrada: Conjunto de treinamento $\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^m$

Saida : Um comitê de modelos de base H

- 1: **Passo 1:** Faça $\mathcal{D}' = \emptyset$
 - 2: **Passo 2:** Divida aleatoriamente $\mathcal{D}$ em K subconjuntos de tamanho igual: $\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_K\}$
 - 3: **for** $k = 1$ to K **do**
 - 4: **Passo 2.1:** Treine os modelos de base do primeiro nível
 - 5: **for** $t = 1$ to T **do**
 - 6: treine um modelo de base h_{kt} a partir $\mathcal{D} \setminus \mathcal{D}_k$
 - 7: **end for**
 - 8: **Passo 2.2:** Aplique cada h_{kt} a cada $\mathbf{x}^{(i)} \in \mathcal{D}_k$, o que resulta em um novo conjunto de exemplos $\mathcal{D}'_k = \{\mathbf{x}'^{(i)}\}$, com $\mathbf{x}'^{(i)} = (h_{k1}(\mathbf{x}^{(i)}), \dots, h_{kT}(\mathbf{x}^{(i)}))$
 - 9: **Passo 2.3:** Faça $\mathcal{D}' = \mathcal{D}' \cup \mathcal{D}'_k$
 - 10: **end for**
 - 11: **Passo 3:** Treine o meta modelo h' em $\mathcal{D}'$
 - 12: **Passo 4:** **for** $t = 1$ to T **do** retreine os modelos de primeiro nível h_t em $\mathcal{D}$
 - 13: **return** $H(\mathbf{x}) = h'(h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_T(\mathbf{x}))$
-

Flexibilidade no Stacking

- Há muita flexibilidade na classificação com Stacking
- Você pode escolher qualquer algoritmo de aprendizado, incluindo métodos de ensemble como bagging ou boosting, para treinar cada “modelo base” no primeiro nível
- O mesmo se aplica ao “meta-modelo” que combina suas previsões
- Regressão logística ou regressão linear são escolhas populares para esse papel
- Aqui está uma abordagem interessante:
 - Em vez de construir um conjunto de dados separado ($\mathcal{D}'$) para treinar o meta-modelo, você pode usar diretamente as características dos dados originais ($\mathcal{D}$)
 - Isso proporciona ao meta-modelo mais contexto sobre as relações entre as características originais e as previsões dos modelos de primeiro nível, potencialmente levando a melhores resultados combinados

- Implementação do Stacking